ITA03-MOBILE COMPUTING

ASSIGNMENT - 4A

ASSIGNMENT-4 – CO4

CO4 Coverage: Different Mobile OS (android/IOS/BB) - Architecture of Android, Blackberry, IOS & windows - Framework of Android - Blackberry - IOS & windows. Psion EPOC -Palm OS -WML (Wireless Markup Language)-J2ME/JME-Symbian Comparing and Contrasting architectures of Android, iOS and Windows - Underlying

OS (Darwin vs. Linux vs. Win 8) - Kernel structure and native level programming - Runtime (Objective-C vs. Dalvik vs. WinRT) - Approaches to power management – Security.

Mobile Application Development Environment – Assignment 4, Part I

SIMATS ENGINEERING · Architectural Analysis and Design Through Structured Reasoning

Course	Mobile Application Development Environment
Course Outcome	CO4 – Analyze mobile operating systems, architectures, runtimes, security, and power management techniques
Nature of Assignment	Individual
Duration	One Week
Total Marks	50
NAME	K.BALAJI
REG.NO	19232180

Purpose of the Assignment

This assignment evaluates your ability to analyse and design a mobile platform solution before building anything. You are expected to demonstrate the following progression:

Interpret the Scenario → Identify Components and Constraints → Design the Architecture or Policy → Document It Precisely → Trace a Scenario → Challenge → Refine

TASK 1 – ARCHITECTURE REVIEW (6 MARKS)

Task 1.1 – Diagnose the Description (2 Marks)

The junior engineer's architecture description contains several technical deficiencies. It does not explain the exact responsibilities of the operating system components.

S.No	Technical deficiency	Explanation	Why it is a problem
1	Missing hardware abstraction layer	The description does not include device drivers or HAL between hardware and upper layers.	Developers cannot understand how applications communicate with hardware.
2	Kernel responsibilities are unclear	It only says that the kernel runs applications.	The kernel actually manages processes, memory, CPU scheduling, device access and system calls.
3	Runtime is not explained	It does not mention whether applications use bytecode, interpretation, JIT or AOT compilation.	Developers cannot determine how application code is executed.
4	Security permissions are incomplete	It does not explain who grants permissions or where they are enforced.	Developers cannot identify how unauthorized access is prevented.
5	Power management is unclear	It only says that the OS saves battery when needed.	There is no information about sleep states, CPU frequency control or background restrictions.
6	Native libraries are missing	The architecture does not identify native libraries and system APIs.	Developers cannot understand how applications access lower-level functionality.
7	Communication boundaries are missing	There is no explanation of communication between applications and OS services.	Engineers cannot identify the system interfaces used by applications.

Conclusion:

The original description is incomplete because it does not define the layers, responsibilities, interfaces and dependencies of a mobile operating system. Another engineer cannot reliably implement or evaluate the architecture using this information.

Task 1.2 – Repair the Description (3 Marks)

Selected Operating System: Android

Android uses a layered architecture in which each layer provides services to the layer above it and depends on the layer below it.

Android Layered Architecture

Layer	**Responsibility**	**Exposes to upper layer**	**Depends on**
1. Hardware	Provides CPU, RAM, display, camera, GPS, sensors and network devices.	Physical device capabilities.	Electronic components.
2. Linux Kernel	Handles process scheduling, memory management, device drivers, networking and system calls.	Kernel system calls and device access.	Hardware and device drivers.
3. Hardware Abstraction Layer (HAL)	Provides standard interfaces for hardware components such as camera, audio and sensors.	Standard hardware service interfaces.	Linux kernel drivers.
4. Native Libraries	Provides SQLite, OpenGL ES, media, graphics and other native functionality.	Native APIs and libraries.	Linux kernel, HAL and system libraries.
5. Android Runtime (ART)	Executes application bytecode using interpretation, JIT and AOT compilation, and manages memory and garbage collection.	Runtime execution and core libraries.	Kernel memory and process services.
6. Application Framework	Provides Activity Manager, Window Manager, Notification Manager, Location Manager and other system services.	High-level application APIs.	ART, native libraries and OS services.
7. Applications	Provides user-facing functions such as messaging, camera, banking and social media.	User interfaces and application functionality.	Application framework and runtime.

Location of Important Components

1. Kernel:

The Linux kernel is located above the hardware and device drivers. It manages CPU scheduling, memory, process execution, networking and hardware communication.

2. Runtime:

ART is located between the native libraries and application framework. It executes application code and manages memory.

3. Security Enforcement:

Android security is enforced at multiple levels:

- Application sandboxing through unique Linux UIDs.
- SELinux mandatory access control.
- Permission checks through Android framework services.
- Package signature verification during installation and updates.
- Keystore-based protection for cryptographic keys.

4. Power Management:

Power management is implemented across the kernel and Android framework.

- Kernel CPU frequency scaling controls processor performance and energy usage.
- Power Management IC controls hardware power states.
- Android Doze restricts background activity during idle conditions.
- JobScheduler and WorkManager allow deferred background work.

Android Architecture Flow

Hardware → Linux Kernel → HAL → Native Libraries and ART → Application Framework → Applications.

Each layer provides defined interfaces to the layer above it and uses services from the layer below it.

Task 1.3 – Design Defence (1 Mark)

The rewritten specification is a precise architectural description because it identifies every layer, its responsibility, the interfaces it exposes and its dependencies. It also explicitly locates the kernel, runtime, security enforcement and power management components. Unlike a marketing overview, it explains the actual technical relationships between the components. Therefore, another engineer can use this specification to understand, evaluate and design a mobile operating system without depending on unclear statements.

TASK 2 – FROM PLATFORM REQUIREMENT TO ARCHITECTURAL COMPARISON (8 MARKS)

Task 2.1 – Model the Requirement (2 Marks)

Scenario: A start-up wants to develop a secure messaging application that receives messages in the background on Android, iOS and Windows devices.

Problem Model

Element	Description
Input	Android, iOS and Windows operating system architectures, kernel structures, runtimes and security mechanisms.
Output	A technical comparison showing the advantages, limitations and suitability of each platform.
Objective	Select a suitable platform architecture for reliable and secure background messaging.
Constraints	Background execution restrictions, security, battery consumption and responsiveness.
Decision	Determine how the messaging application should handle background message delivery on each platform.
Required information	Kernel family, runtime, application framework, IPC, process isolation, background execution and notification services.

Explanation:

The application must receive messages even when the screen is switched off. Therefore, the team must understand how each operating system schedules background processes, handles network events and delivers notifications.

Task 2.2 – Formulate a Hypothesis (1 Mark)

Hypothesis:

The background execution and process-management model will have the greatest effect on message delivery because the operating system controls whether an application remains active, is suspended or is terminated, directly affecting its ability to receive messages.

Task 2.3 – Build the Comparison (3 Marks)

Android vs iOS vs Windows Architecture

Comparison	Android	iOS	Windows

Kernel family	Linux kernel with a modular monolithic structure.	Darwin XNU hybrid kernel combining Mach and BSD components.	Windows NT hybrid kernel architecture.
Kernel responsibilities	Process scheduling, memory management, networking and device drivers.	Memory management, scheduling, IPC and hardware management.	Process scheduling, memory management, security and device management.
Kernel programming interface	Linux system calls and Android-specific interfaces.	Darwin system interfaces, POSIX-related APIs and Mach interfaces.	Windows Native API and system service interfaces.
Runtime environment	Historically Dalvik VM; modern Android uses ART.	Objective-C runtime for dynamic method dispatch, object management and runtime services.	WinRT provides a modern application API and runtime object model.
Application framework	Android SDK, Activity Manager, Notification Manager and other services.	Cocoa Touch, UIKit and Foundation frameworks.	Windows Runtime APIs and Windows application frameworks.
Application isolation	Each application normally receives a separate Linux UID and sandbox.	Applications run in separate sandboxes with restricted entitlements.	Applications use package identity, AppContainer isolation and capabilities.
IPC mechanism	Binder IPC.	XPC and Mach ports.	COM/WinRT communication and other Windows IPC mechanisms.
Background execution	Services, WorkManager, foreground services and push notifications.	Background execution is restricted; APNs can wake or notify an application through supported mechanisms.	Background tasks and push notification services, subject to platform restrictions.
Messaging consequence	Flexible background processing, but battery	Push notifications are important because	Background tasks and push notifications

	and background limits must be managed.	continuous background execution is restricted.	support controlled message delivery.

Analysis of Each Platform

1. Android

Android uses the Linux kernel. Its Binder IPC mechanism allows applications and system services to communicate. Applications are isolated using Linux UIDs and SELinux.

Android supports background services, WorkManager and Firebase Cloud Messaging (FCM). However, background execution limits and Doze can delay some work.

Consequence: Android provides flexible background processing, but developers must follow power-management restrictions.

2. iOS

iOS uses the Darwin operating system with the XNU hybrid kernel. The Objective-C runtime supports dynamic method dispatch and object management. Applications communicate with system services using mechanisms such as XPC and Mach ports.

Apple Push Notification service (APNs) delivers notifications without requiring an application to remain continuously active.

Consequence: The messaging application should use APNs for reliable notification delivery instead of continuously running a background process.

3. Windows

Windows uses the Windows NT kernel architecture. WinRT provides application APIs and a modern runtime object model.

Applications use package isolation and controlled background tasks. Push notification services can deliver notifications when an application is not actively running.

Consequence: The application must use supported background tasks and push notification mechanisms instead of depending on continuous execution.

Final Comparison Decision

Android is suitable when flexible background processing is required. iOS is suitable when push-based background notifications are acceptable. Windows can support controlled background execution through its application platform.

A common messaging architecture should use a secure server-side message queue and platform-specific push notification services.

Task 2.4 – Desk-Check (1 Mark)

Scenario: One incoming message arrives while the screen is switched off.

Platform 1: Android

Step	Component	Operation
1	Network hardware	Receives incoming network packets.
2	Linux kernel	Handles network communication through networking drivers and TCP/IP.
3	Android system services	Network and notification-related services coordinate system activity.
4	FCM service	Delivers a push message through Google Play services when available.
5	Application process	The system may start or wake the application process for an eligible message callback.
6	ART runtime	Executes the application's callback code.
7	Application framework	Displays a notification using Notification Manager.

Result: The user receives a notification even when the screen is off, subject to connectivity, notification settings and background restrictions.

Platform 2: iOS

Step	Component	Operation
1	Network hardware	Receives the push notification data.
2	Darwin kernel	Handles network and hardware communication.
3	iOS notification services	Processes the notification received through APNs.
4	APNs	Provides notification delivery to the device.
5	Application lifecycle	The OS may notify the application through an eligible background notification callback.
6	Objective-C runtime	Executes the relevant application code if the OS grants background execution.
7	Notification framework	Displays the notification to the user.

Result: iOS can display a push notification without requiring the application to remain continuously active.

Task 2.5 – Challenge Your Comparison (1 Mark)

Changed condition: The application is terminated by the operating system because of memory pressure.

Prediction

Android and iOS may behave differently depending on their process-management policies and notification mechanisms. In both cases, ordinary application code cannot be assumed to continue executing after process termination.

Responsible Architectural Components

- Android: Activity Manager, process management, FCM and ART.
- iOS: Application lifecycle management, APNs and notification services.

Modified Trace

Step	Android	iOS
1	Application process is killed.	Application process is terminated.
2	FCM push message reaches the device.	APNs push notification reaches the device.
3	System may start the eligible application process for a supported message callback.	System may display the notification without launching the application.
4	ART initializes if the application is started.	Objective-C runtime initializes if application code is started.
5	Application handles the event if permitted.	Application handles an eligible background callback if permitted.

Conclusion:

The prediction is correct: terminating the application process does not necessarily prevent push notification delivery. However, background callbacks are subject to OS restrictions. Force-stopping an Android application or disabling notifications can change the result.

TASK 3 – RUNTIME ENVIRONMENT ANALYSIS (9 MARKS)

Task 3.1 – Define the Execution Model (1 Mark)

Android vs iOS vs Windows Runtime

Feature	Android (Dalvik / ART)	iOS (Objective-C runtime)	Windows (WinRT)
Application code	Historically DEX bytecode; modern Android applications commonly use DEX executed by ART.	Native machine-code binaries.	Native binaries or supported managed application components depending on the development technology.
Loader	Android application process and ART runtime.	iOS process loader and dynamic linker.	Windows loader and runtime infrastructure.
Execution	Dalvik used bytecode interpretation and JIT; ART uses interpretation, JIT and AOT compilation.	Native instructions are executed by the processor.	Native code executes directly; managed technologies can use their respective runtime compilation models.
Compilation	Runtime and installation/profile-guided compilation.	Mainly ahead-of-time compilation.	Depends on the application technology and compiler.
Memory management	Garbage collection in ART.	Automatic Reference Counting (ARC) in Swift and Objective-C development.	Depends on the programming language and application technology.

Conclusion: Android uses a managed runtime, iOS primarily executes native code with ARC-based object lifetime management, and Windows supports different application execution technologies through its platform APIs.

Task 3.2 – Develop the Launch Sequence (2 Marks)

Selected Platform: Android

Application Launch Sequence

Step	Operation	Component	Requirement
1	User taps the application icon.	Launcher	Application icon must be installed.
2	Launch request is sent to Android.	Activity Manager	Application launch intent must be available.

3	Android checks whether the application process already exists.	Activity Manager	Application process information must be available.
4	If required, Android creates a new application process.	Zygote	Zygote process must already be running.
5	Application code and resources are loaded.	ART and class loader	Application package must be accessible.
6	Runtime initialization takes place.	ART	Runtime libraries and process memory must be initialized.
7	Main Activity is created.	Application framework	Activity component must be registered.
8	Lifecycle callbacks are executed.	Activity framework	Application process and runtime must be ready.
9	Layout and UI components are measured and drawn.	Window Manager and rendering system	Valid UI layout and display services must exist.
10	First screen appears.	Display subsystem	Rendering and display hardware must be operational.

Purpose of the sequence:

The sequence ensures that the operating system creates the required process, initializes the runtime, loads application components and prepares the user interface before displaying the first screen.

Task 3.3 – Compare Runtime Strategies (4 Marks)

1. Interpretation, JIT and AOT

Interpretation:

The runtime reads intermediate instructions and executes them without first compiling the entire application into native machine code.

Just-in-Time Compilation (JIT):

JIT compilation converts frequently executed code into machine instructions during application execution.

Ahead-of-Time Compilation (AOT):

AOT compilation converts application code into native machine instructions before execution.

Platform	Runtime strategy
Android	Dalvik historically used interpretation and JIT. ART supports interpretation, JIT and profile-guided AOT compilation.
iOS	Native binaries are mainly compiled ahead of time using Apple's development toolchain.
Windows	Native applications use compiled machine code; managed application technologies can use runtime compilation.

2. Memory Management

Platform	Memory management	Effect on responsiveness and battery
Android	ART garbage collection.	Automatically reclaims unreachable objects, but GC can introduce pauses and CPU overhead.
iOS	ARC in Swift and Objective-C.	Releases objects when strong reference counts reach zero, reducing tracing GC overhead but requiring careful reference management.
Windows	Depends on the language; native applications commonly use explicit or smart-pointer-based lifetime management, while managed technologies can use garbage collection.	Behaviour depends on the application runtime and memory-management strategy.

3. Access to Native Code and Kernel Services

Android:

- Java/Kotlin applications use Android framework APIs.
- JNI allows communication with native C/C++ code.
- Native libraries communicate with OS services through supported system interfaces.
- Linux kernel system calls provide low-level services.

iOS:

- Swift and Objective-C applications use Apple frameworks.
- Objective-C runtime provides dynamic method dispatch.
- Native APIs provide access to device services.

- Darwin and Mach interfaces provide lower-level OS services.

Windows:

- Applications use WinRT APIs and Windows SDK interfaces.
- Native applications can use supported Windows system APIs.
- Windows NT provides lower-level process, memory and hardware services.

4. Runtime Effect on Performance

Parameter	Android ART	iOS	Windows
Startup time	May require runtime initialization and class loading.	Native execution can provide efficient startup.	Depends on application technology and runtime initialization.
Memory footprint	Runtime and managed heap require memory.	ARC manages object lifetime; application memory still depends on allocation.	Depends on the application framework and runtime.
Portability	Android bytecode can execute across supported Android devices.	Native applications are designed for Apple platforms.	Depends on supported Windows devices and application technology.
Performance	JIT/AOT optimization improves frequently executed code.	Native compilation offers direct machine-code execution.	Native applications can provide direct execution performance.

Which Runtime Is Most Suitable for a Low-Memory Device?

A carefully designed native application with controlled memory allocation is suitable for a low-memory device. Among the three approaches, an appropriately optimized native iOS-style execution model can reduce managed-runtime overhead because it does not require ART's managed heap.

However, ARC also consumes memory, and it does not automatically guarantee lower memory usage. Actual suitability depends on the application, framework and device.

Task 3.4 – Desk-Check (1 Mark)

Selected platform: Android

Cold Start

A cold start occurs when the application process is not running.

Step	Operation	Status
1	User taps the application icon.	Executed
2	Activity Manager checks process status.	Executed
3	Zygote creates the application process.	Executed
4	ART initializes.	Executed
5	Application classes and resources are loaded.	Executed
6	Main Activity is created.	Executed
7	UI is rendered.	Executed

Warm Start

A warm start occurs when the application process still exists but its Activity needs to be brought back to the foreground.

Step	Operation	Status
1	User taps the application icon.	Executed
2	Activity Manager checks process status.	Executed
3	New process creation through Zygote.	Skipped
4	ART initialization.	Usually skipped
5	Existing application objects and resources are reused where possible.	Partly reused
6	Activity is resumed or recreated if necessary.	Executed as required
7	UI is brought to the foreground.	Executed

Conclusion: A warm start is generally faster because the application process and initialized runtime may already exist.

Task 3.5 – Analyse the Dependency (1 Mark)

The Android launch sequence depends on the correct initialization order of OS components.

First, the kernel must provide process scheduling and memory allocation. Next, Zygote creates the application process. ART must then initialize to load and execute application code.

After runtime initialization, the application framework can create Activity components and execute lifecycle callbacks.

If the kernel cannot allocate memory or schedule the process, the application cannot start. If ART is not initialized, application code cannot execute. Therefore, kernel services and runtime initialization must be completed before framework callbacks and application code can run.

TASK 4 – SECURITY FORENSICS (7 MARKS)

Task 4.1 – Locate the Defects (2 Marks)

The SecureNotes application contains four important security defects.

No.	Original design decision	Security defect	OS security mechanism bypassed
1	Requests every permission during installation.	Unnecessary access to sensitive resources.	Runtime permission model and least privilege.
2	Saves login token in a file readable by other applications.	Other applications may access the token if storage is exposed.	Application sandboxing and secure storage.
3	Accepts updates from any source without verification.	Malicious or modified application packages may be installed.	Code signing and package verification.
4	Uses unencrypted server communication.	Attackers can intercept or modify data.	TLS transport protection.

Explanation

1. Excessive Permissions:

The application should request only necessary permissions. Sensitive permissions should be requested when the relevant feature is used.

2. Insecure Token Storage:

Authentication tokens must not be stored in publicly accessible files. Secure storage and application-private storage should be used.

3. Unverified Updates:

Updates must be signed by a trusted developer identity. The operating system should verify package signatures and update integrity.

4. Unencrypted Communication:

Plain HTTP communication exposes sensitive information. HTTPS with TLS must be used to protect data in transit.

Task 4.2 – Construct an Attack Scenario (2 Marks)

Scenario: Malicious Application Steals SecureNotes Login Token

Attacker: A malicious application installed on the same mobile device.

Defects used:

1. Login token is stored in a file accessible to other applications.
2. Server communication uses unencrypted HTTP.

Attack Sequence

Step	Attack action	Result
1	User installs SecureNotes.	Application is installed with insecure storage design.
2	User logs in to SecureNotes.	Authentication token is generated.
3	Token is saved in an accessible file.	Confidentiality is lost because unauthorized access becomes possible.
4	Malicious application reads the exposed token where the file permissions allow it.	Attacker obtains the authentication token.
5	Attacker uses the token to access the server.	Unauthorized access to user information may occur.
6	Attacker monitors unencrypted network traffic.	Additional sensitive information may be intercepted or modified.

First Point of Security Failure

The first confidentiality failure occurs at Step 3, when the authentication token is stored without adequate protection.

The unencrypted network communication creates an additional point of exposure.

Impact:

- Unauthorized access to user accounts.
- Exposure of confidential notes.
- Possible modification of user information.

- Loss of user privacy and trust.

Task 4.3 – Repair the Design (2 Marks)

Selected Operating System: Android

Only the defective security decisions are corrected.

Defect	**Required repair**	**Android mechanism**	**Security benefit**
Excessive permissions	Request permissions only when required.	Android runtime permissions.	Reduces unnecessary access.
Insecure token storage	Store authentication tokens in encrypted application-private storage and protect encryption keys using Android Keystore.	Application sandbox and Android Keystore.	Protects sensitive authentication data.
Unverified updates	Accept updates only from trusted signed distribution channels and verify package signatures.	Android package manager and app-signing verification.	Prevents unauthorized package replacement.
Unencrypted communication	Replace HTTP with HTTPS and TLS certificate validation.	Android networking and TLS security mechanisms.	Protects confidentiality and integrity during transmission.

Corrected Security Architecture

1. The application requests permissions only when the user accesses a feature.
2. Authentication tokens are stored in encrypted private storage.
3. Cryptographic keys are protected using Android Keystore.
4. Application updates are verified through package-signing mechanisms.
5. All server communication uses HTTPS with TLS.
6. Android sandboxing and SELinux restrict unauthorized access.

Result: The repaired design provides multiple layers of security instead of relying on a single protection mechanism.

Task 4.4 – Regression Check (1 Mark)

Legitimate Behaviour: Reading Contacts After User Approval

The original application may legitimately require access to contacts to allow the user to select a person.

Regression test:

Step	Action	Expected result
1	User opens the contact-selection feature.	Application requests contact permission.
2	User grants permission.	OS allows authorized contact access.
3	Application reads the contacts.	Contact information is displayed.
4	User denies permission.	Application does not access contacts and displays an appropriate message.

Conclusion:

The security repairs preserve legitimate contact access because the application still receives permission when the user approves it. Unnecessary permissions and unauthorized access remain restricted.

TASK 5 – POWER MANAGEMENT REASONING (7 MARKS)

Task 5.1 – Design the Power-State Policy (3 Marks)

Scenario: A fitness application keeps the processor awake permanently, causing excessive battery consumption.

Proposed Power-State Model

The device uses three states:

1. Active State
2. Idle State
3. Suspended State

State-Transition Table

State	Hardware and kernel activity	Entry condition	Exit condition	Allowed applications/services
Active	CPU operates normally, display is active and sensors provide readings.	User opens the application or an active	User becomes inactive or screen turns off.	Foreground applications, required services and system processes.

		tracking session starts.		
Idle	CPU enters lower-power operation when possible; display may be off and unnecessary activities are restricted.	Screen remains off and the device is stationary.	User interaction, scheduled task or important event.	Limited background services, alarms and eligible scheduled tasks.
Suspended	CPU execution is largely paused; selected hardware components may remain active for wake events.	No immediate user activity or scheduled work is required.	Incoming call, wake event, user interaction or eligible system event.	Essential system services and hardware-supported wake mechanisms.

State Transition Diagram

Active → Idle → Suspended

Suspended → Idle → Active

Explanation

Active State:

The fitness application can collect sensor data at the required frequency. The processor remains available for immediate work.

Idle State:

The application reduces unnecessary sensor sampling and batches data. The system can reduce CPU activity and network wake-ups.

Suspended State:

The device enters a low-power state. The application should not continuously hold a CPU wakelock. Important system events can wake the device.

Conclusion:

This policy reduces unnecessary processor activity while allowing essential fitness tracking operations.

Task 5.2 – Change the Assumption (2 Marks)

Changed requirement: The application must receive urgent push notifications while the device is suspended.

Problem in the Previous Policy

The previous policy assumes that the device can remain suspended without interruption. This is insufficient when urgent notifications must be received.

Required Platform Mechanisms

1. Push-Based Wake-Up:

A push notification service can deliver an important notification through supported OS and network mechanisms.

2. Wake Locks:

A partial wakelock can temporarily keep the CPU active when essential processing is required. It should be held only for the minimum necessary duration.

3. Scheduled Background Tasks:

JobScheduler or WorkManager can perform deferred operations when the required conditions are met.

4. Batching:

Sensor readings and network operations can be grouped to reduce repeated wake-ups.

Corrected Policy

1. Keep the device in suspended mode whenever possible.
2. Use push notifications for urgent events.
3. Wake the CPU only when necessary.
4. Process urgent data quickly.
5. Release wakelocks immediately after processing.
6. Schedule non-urgent work for suitable maintenance windows.

Conclusion:

The application must stop continuously keeping the processor awake. Push-based wake-up and short-duration processing allow urgent notifications while preserving battery life.

Task 5.3 – Change the Requirement (2 Marks)

New requirement: Battery-saver mode activates automatically when battery charge reaches 15%.

Battery-Saver Behaviour

Layer	**Changes when battery saver activates**

Kernel level	Reduces CPU performance where supported, uses lower-power scheduling and limits unnecessary wake-ups.
Runtime level	Reduces unnecessary background execution and resource-intensive processing.
Application level	Reduces sensor sampling frequency, delays non-urgent uploads and batches network requests.

User-Visible Trade-Off

Trade-off: Fitness tracking updates may become less frequent.

For example:

- Normal mode: Sensor sampling every 5 seconds.
- Battery-saver mode: Sensor sampling every 30 seconds.

Justification

The reduced sampling frequency saves energy and helps the device operate for a longer period. Essential tracking information can still be collected, while non-urgent updates are delayed.

Conclusion:

Battery-saver mode provides a balance between application functionality and energy consumption when the battery level reaches 15%.

TASK 6 – DESIGN CHALLENGE (13 MARKS)

Platform Strategy for a Field-Data Collection System

Scenario

RuralCare collects patient records using three classes of devices:

- Class A – Modern Android smartphones.
- Class B – iPhones.
- Class C – Legacy feature phones supporting J2ME/JME applications or WML browsers.

The system must support offline data collection, ten-hour battery operation, secure patient records and reliable synchronisation.

Stage 1 – Interpret the Problem (2 Marks)

Problem Model

Element	Description
Input	Patient information, device class, battery level, network availability and collected field records.
Output	Securely stored patient records and correctly synchronised server records.
Objective	Collect and synchronise patient information without data loss or duplication while maintaining a ten-hour battery life.
Constraints	Offline operation, limited device storage, security, battery capacity and different mobile OS architectures.
Decisions	Platform strategy, offline storage, encryption method, synchronisation protocol and power-management policy.

Explanation

The system must continue working even when internet connectivity is unavailable. Patient information must be protected if a device is lost.

The system must also identify previously uploaded records to avoid duplication during synchronisation.

Conclusion: The solution requires a platform-specific architecture with a common backend and a reliable offline-first data-management strategy.

Stage 2 – Propose the Strategy (2 Marks)

Selected Strategy: Hybrid Multi-Platform Architecture

The proposed strategy uses native applications for Android and iOS and a lightweight web-based fallback for legacy devices.

Device class	Selected strategy	Justification
Class A – Android	Native Android application using Kotlin/Java.	Supports encrypted local storage, Android Keystore, background tasks and flexible power management.
Class B – iPhone	Native iOS application using Swift.	Supports Keychain, Data Protection, background task APIs and Apple security mechanisms.
Class C – Legacy feature phones	J2ME application where supported; WML browser fallback for simpler devices.	Supports basic data entry where smartphone applications and modern runtimes are unavailable.

Why This Strategy?

Android:

Android provides application-private storage, database support and background scheduling. It is suitable for offline data collection.

iOS:

iOS provides strong application sandboxing, Keychain and Data Protection mechanisms. It is suitable for sensitive patient information.

Legacy Devices:

J2ME applications can provide basic offline record entry where persistent storage is available. WML browsers can provide a simpler form-based interface where local application storage is limited.

Backend Strategy

All three classes communicate with a common secure backend through a standard API when connectivity is available.

The backend manages:

- Patient records.
- User authentication.
- Record identifiers.
- Synchronisation status.
- Conflict detection.
- Audit history.

Conclusion:

A hybrid multi-platform architecture is suitable because it supports modern smartphone security while maintaining limited access for legacy feature phones.

Stage 3 – Identify the Critical Information (2 Marks)

Constraint Comparison Table

Parameter	**Class A – Android**	**Class B – iOS**	**Class C – Legacy**
Runtime	ART	Native Swift/Objective-C runtime	J2ME virtual machine or WML browser
Security mechanisms	Linux UID sandbox, SELinux, Keystore and runtime permissions	Application sandbox, Keychain and Data Protection	Limited application security depending on

			device and MIDP implementation
Power management	Doze, WorkManager and wakelocks	Background task restrictions and system-managed power controls	Limited power-management APIs
Offline storage	Room/SQLite and encrypted files	Core Data/SQLite with protected storage	RMS for J2ME where supported; limited browser storage for WML
Connectivity	Wi-Fi, mobile data and Bluetooth	Wi-Fi and mobile data	Device-dependent GPRS or other supported mobile network

Information Retained for Synchronisation

Information	**Purpose**
Unique record ID	Identifies every patient record and prevents duplicate creation.
Patient ID	Connects records to the correct patient.
Device ID	Identifies the device that created the record.
Creation timestamp	Records when the information was collected.
Last modified timestamp	Helps identify changes and resolve conflicts.
Synchronisation status	Identifies pending, uploaded or failed records.
Version number	Detects whether a record has been updated.
Server acknowledgement ID	Confirms that the server successfully received a record.
Encryption metadata	Identifies the encryption format and key version needed for decryption.

Why Is This Information Required?

Unique IDs prevent duplicate records. Timestamps and version numbers help resolve conflicts. Synchronisation status prevents already uploaded records from being sent repeatedly.

Server acknowledgements ensure that records are not deleted locally before successful upload confirmation.

Stage 4 – Design the Complete Solution Specification (4 Marks)

4.1 Overall System Architecture

The system contains four major layers.

Layer	Components	Responsibility
Presentation layer	Patient entry forms, login screen and record status screen.	Accepts field-worker input and displays results.
Application/domain layer	Validation, patient record management and synchronisation rules.	Processes records and applies business rules.
Data layer	Encrypted local database, record queue and encryption management.	Stores patient information securely during offline operation.
OS services and communication layer	Android/iOS security services, network APIs and backend server.	Provides storage protection, network communication and OS functionality.

Architecture Flow

Field Worker → User Interface → Validation Layer → Encryption Service → Local Database → Synchronisation Manager → Secure API → RuralCare Server.

4.2 Data Capture and Offline Storage Design

Step 1: Field worker logs in using authenticated credentials.

Step 2: Application downloads the required patient information when network connectivity is available.

Step 3: Field worker enters new patient details.

Step 4: Application validates mandatory fields such as patient ID, name and required medical information.

Step 5: Application generates a unique record ID.

Step 6: Patient data is encrypted before being written to local storage.

Step 7: The record is stored with a PENDING synchronisation status.

Step 8: The application continues collecting records without internet access.

4.3 Security Design

Security requirement	Class A – Android	Class B – iOS	Class C – Legacy

Data at rest	Encrypted database with keys protected by Android Keystore.	Protected local storage with keys stored in Keychain.	Application-private RMS where supported; avoid storing sensitive records on devices without adequate protection.
Data in transit	HTTPS with TLS.	HTTPS with TLS.	HTTPS where supported; otherwise use a restricted, approved gateway or do not transmit sensitive data.
Authentication	Secure token-based authentication.	Secure token-based authentication.	Restricted user credentials and server-side session controls where supported.
Lost device	Remote logout, credential revocation and remote wipe where supported.	Find My/device management and remote wipe where supported.	Revoke device credentials and avoid retaining sensitive patient data locally.
Access control	Android sandbox and permissions.	iOS sandbox and entitlements.	Limited device-level isolation; server-side controls are essential.

Lost Device Protection

If a device is lost:

1. The worker reports the loss.
2. The administrator disables the device account.
3. Refresh tokens and device credentials are revoked.
4. Remote wipe is initiated where supported.
5. Server access is blocked for the lost device.
6. Previously synchronised records remain available on the server.

Important: A legacy device that cannot securely store patient information must use a minimal-data approach or be excluded from storing sensitive records offline.

4.4 Power Management Policy for a Ten-Hour Shift

Battery condition	**Sampling policy**	**Synchronisation policy**

80–100%	Normal record collection.	Upload when Wi-Fi or suitable connectivity is available.
50–79%	Normal collection with batching.	Batch records during suitable network periods.
20–49%	Reduce unnecessary background activity.	Upload pending records periodically.
Below 20%	Prioritise essential data collection.	Defer non-urgent uploads where safe.
Critical battery	Save current work and protect queued records.	Synchronise when charging or connectivity becomes available.

Additional power-saving measures:

- Turn off unnecessary GPS usage.
- Batch network requests.
- Avoid continuous CPU wakelocks.
- Use scheduled background synchronisation.
- Reduce unnecessary screen brightness.
- Avoid repeatedly polling the server.

4.5 Synchronisation Procedure

Normal Synchronisation Steps

1. Check network connectivity.
2. Authenticate the device with the server.
3. Retrieve pending local records.
4. Check unique record IDs and version numbers.
5. Encrypt and transmit records through HTTPS.
6. Server validates the records.
7. Server checks whether each record already exists.
8. New records are inserted and updated records are reconciled.
9. Server sends an acknowledgement.
10. Local synchronisation status changes from PENDING to SYNCED.
11. The application retains records according to the approved retention policy.

Handling Interrupted Synchronisation

If the network disconnects during upload:

1. Stop the current transfer.
2. Keep all unacknowledged records in the local database.
3. Mark the affected records as PENDING or RETRY.
4. Do not delete local records.
5. Reconnect when network access returns.
6. Retry using the same unique record IDs.
7. Server checks existing IDs before inserting records.
8. Confirm acknowledgement before marking records as SYNCED.

Result: Repeated uploads do not create duplicate patient records.

4.6 Failure Handling

Failure condition	**System response**
No internet connection	Save records locally in encrypted storage.
Low battery	Reduce sampling and postpone non-urgent work.
Application crash	Recover pending records from persistent storage.
Duplicate record	Detect using unique record ID.
Authentication failure	Stop synchronisation and request valid authentication.
Server unavailable	Retain records locally and retry later.
Storage full	Display warning and prevent incomplete record saving.
Device lost	Revoke credentials and initiate remote protection measures.
Legacy device lacks secure storage	Avoid storing sensitive patient data locally.

Conclusion:

The architecture provides secure data collection, offline storage, battery-aware operation and reliable synchronisation across three different device classes.

Stage 5 – Desk-Check (1 Mark)

Scenario

A Class A worker is offline for six hours. The battery falls to 20%, and the phone is lost before synchronisation.

Step-by-Step Trace

Step	Event	System action	Result
1	Worker logs in.	Authentication is completed.	Worker can access the application.
2	Network becomes unavailable.	Application enters offline mode.	Data collection continues.
3	Worker creates patient records.	Application validates and encrypts records.	Records are saved securely.
4	Six hours pass.	Records remain in encrypted local storage.	No server synchronisation occurs.
5	Battery reaches 20%.	Battery-saving policy activates.	Non-essential operations are reduced.
6	Phone is lost.	Worker reports the loss.	Administrator begins incident response.
7	Credentials are revoked.	Server rejects future access from the device.	Unauthorized server access is restricted.
8	Remote wipe is attempted.	Device management initiates remote protection if supported.	Local data may be removed if the device reconnects and the wipe succeeds.

Information Exposure and Recovery

Question	Result
Is patient data exposed?	It remains encrypted at rest. Exposure risk increases if the device is unlocked or compromised.
Is local data lost?	It may become inaccessible if the device is permanently lost or wiped.
Can unsynchronised records be recovered?	Not guaranteed if the only copy was on the lost device.

Can synchronised records be recovered?	Yes, they remain available on the server.
Can the attacker access the server?	Revoked credentials restrict further access.

Conclusion:

Encryption protects stored records, and credential revocation protects server access. However, unsynchronised records stored only on the lost device cannot be guaranteed recoverable. Therefore, periodic synchronisation and device-loss controls are important.

Stage 6 – Attack Your Own Design (2 Marks)

Adverse Condition 1 – Stolen Unlocked Phone

Challenge:

An attacker steals a phone while the application is unlocked.

Prediction:

The attacker may access the currently available application information.

Test Result:

The application requires re-authentication for sensitive operations, uses encrypted storage and limits session access. However, information already visible on the screen may remain exposed.

Required refinement:

Use automatic session locking, biometric or PIN re-authentication for sensitive records, and remote credential revocation.

Adverse Condition 2 – Corrupted Synchronisation During Transfer

Challenge:

The network disconnects after some records are transmitted but before acknowledgement is received.

Prediction:

Some records may be uploaded while others remain pending.

Test Result:

The system retains unacknowledged records and retries using unique record IDs. The server checks existing IDs before inserting records.

Result:

Duplicate record creation is prevented, and incomplete synchronisation can be resumed.

Adverse Condition 3 – Legacy Device Without Secure Storage

Challenge:

A Class C feature phone does not provide suitable encryption or protected local storage.

Prediction:

Patient information stored locally may be exposed if the device is lost.

Test Result:

The system avoids storing full sensitive patient records on unsupported devices. It restricts the legacy workflow to minimal non-sensitive information and requires secure server-side access.

Required refinement:

If secure offline storage is mandatory, the device must be replaced with a platform capable of providing adequate encryption and isolation.

Final Evaluation

Adverse condition	**Does the design survive?**	**Required action**
Stolen unlocked phone	Partially	Re-authentication and session locking.
Interrupted synchronisation	Yes	Retry with unique IDs and server acknowledgements.
Legacy device without secure storage	Partially	Restrict sensitive offline storage or replace the device.

Critical Design Decision:

The most important design decision is encrypting patient data before local storage and using unique record IDs during synchronisation.

Overall Conclusion:

The proposed design handles offline operation, duplicate prevention, interrupted synchronisation and device loss. However, a device without secure local storage cannot safely support full offline patient-record storage. This limitation must be addressed before deployment.

FINAL CONCLUSION

This assignment analysed mobile operating system architectures, runtimes, security mechanisms and power management techniques.

ITA03-MOBILE COMPUTING

ASSIGNMENT-4B

ASSIGNMENT 4 – PART II

MOBILE SYSTEMS ENGINEERING: ANALYSE, INSTRUMENT, TEST, DEBUG & OPTIMISE

Course Outcome: CO4 – Analyse mobile operating systems, architectures, runtimes, security and power management techniquesNature: IndividualDuration: One WeekTotal Marks: 50Environment: Android Studio / Emulator / Physical device with adb, Linux shell, Python or Kotlin/Java (student's choice)Prerequisite: Completion of CO4 – Assignment Part I

Android, iOS and Windows use different kernel structures, application frameworks and runtime environments. These differences influence application execution, background processing, memory management and security.

The SecureNotes analysis demonstrated the importance of permissions, sandboxing, code signing, secure storage and encrypted communication. The power-management analysis showed how active, idle and suspended states help reduce battery consumption.

Finally, the RuralCare design demonstrated how different mobile platforms can work together using secure offline storage, power-aware data collection and reliable synchronisation.

Therefore, a successful mobile system must balance performance, security, battery life, reliability and device limitations rather than depending on a single operating system feature or technology.

TASK 1 – ANALYSE AND VALIDATE A MOBILE OPERATING SYSTEM MODEL (8 MARKS)

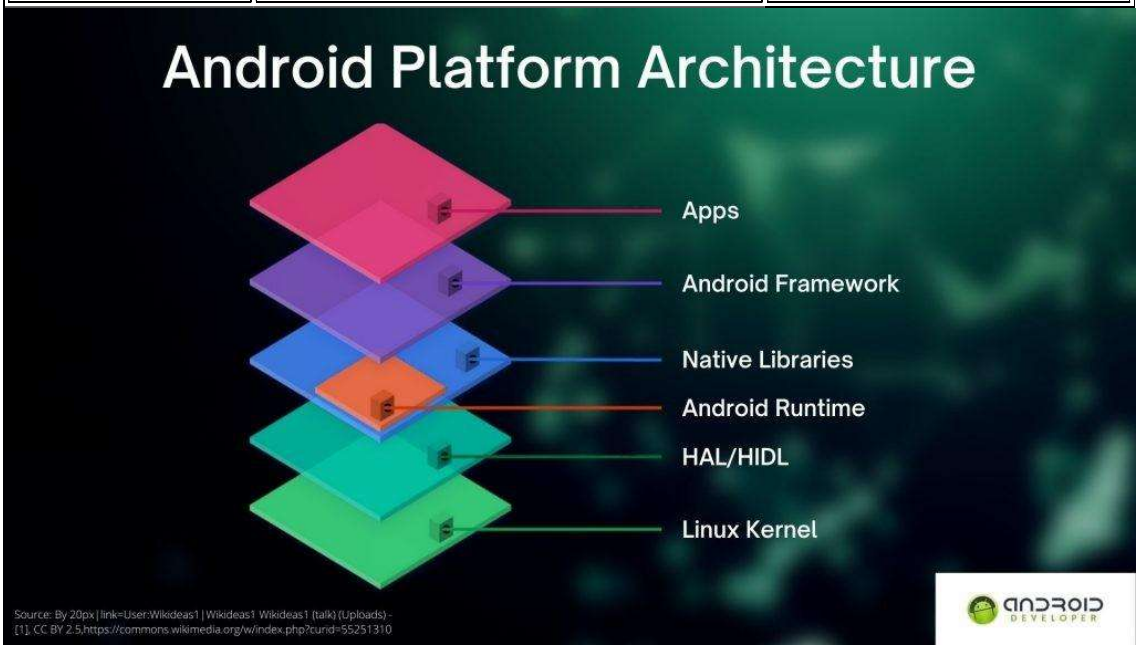
1.1 Analyse the OS Architecture (3 Marks)

A. Android Architecture

Android follows a layered architecture consisting of the Linux kernel, HAL, native libraries, Android Runtime, application framework and applications.

Layer	**Responsibility**	**Example**
Applications	Provides user-facing functionality.	WhatsApp, Camera, Maps
Application Framework	Provides APIs and system services.	Activity Manager, Notification Manager

Android Runtime	Executes application code and manages memory.	ART, garbage collector
Native Libraries	Provides graphics, database and multimedia functionality.	SQLite, OpenGL ES
HAL	Provides standard interfaces to hardware.	Camera HAL, Audio HAL
Linux Kernel	Manages processes, memory, drivers and networking.	Scheduler, memory manager
Hardware	Provides physical device resources.	CPU, RAM, GPS, camera

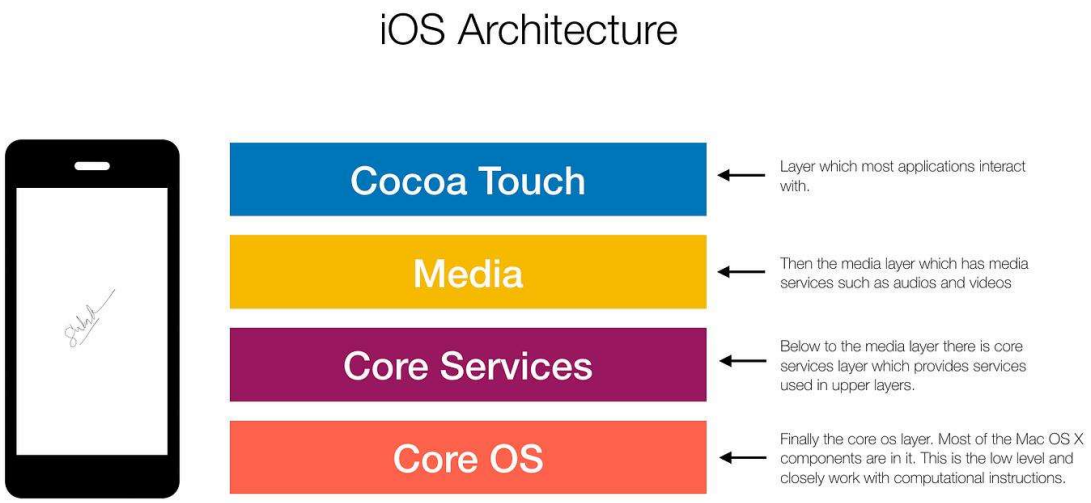


B. iOS Architecture

iOS uses four major software layers.

Layer	**Responsibility**	**Examples**
Cocoa Touch	Provides user interface and application lifecycle management.	UIKit, notifications
Media	Handles graphics, audio and video.	Core Animation, AVFoundation
Core Services	Provides data, networking and location services.	Foundation, Core Location

Core OS	Provides kernel, drivers, security and low-level services.	Darwin, XNU, networking



All Rights Reserved To Anuradh Caldera

C. Process Model Comparison

Feature	**Android**	**iOS**
Process creation	Zygote creates application processes.	launchd manages system services and application launch.
Process management	Activity Manager and Linux kernel manage application processes.	iOS process and lifecycle management controls application execution.
IPC	Binder IPC.	Mach ports and XPC.
Application lifecycle	Activity and Service lifecycle.	UIApplication and UIViewController lifecycle.
Memory management	Android uses process importance and memory pressure policies.	iOS uses system-managed memory pressure and application lifecycle policies.

D. Inter-Process Communication

Android – Binder

Binder is the main Android IPC mechanism. It allows an application to communicate with system services and other permitted processes.

Example:

- An application requests location information.
- The request is sent to a system service through Binder.
- The service communicates with the required hardware subsystem.
- The result is returned to the application.

iOS – XPC and Mach Ports

XPC provides inter-process communication between applications and system services. Mach ports provide lower-level message communication.

E. Component Lifecycle

Android Activity Lifecycle

onCreate()

|

onStart()

|

onResume()

|

Application Running

|

onPause()

|

onStop()

|

onDestroy() (not guaranteed)

iOS View Controller Lifecycle

viewDidLoad()

|

viewWillAppear()

|

viewDidAppear()

|

View Active

|

viewWillDisappear()

|

viewDidDisappear()

F. Commands to Collect Android Evidence

Run these commands in the Android Studio Terminal or Windows CMD after connecting an emulator.

1. Display connected devices

adb devices

Expected output:

List of devices attached

emulator-5554 device

2. Display running processes

adb shell ps -A

3. Display Android memory information

adb shell dumpsys meminfo

4. Display Activity Manager process information

adb shell dumpsys activity processes

5. Display Binder information

adb shell dumpsys activity services

6. Display Android version

adb shell getprop ro.build.version.release

Interpretation

These commands provide evidence about the Android process model, memory consumption, application services and operating system version.

Binder is used for communication between application processes and Android system services. Process information helps determine which applications are active and which may be candidates for termination.

1.2 Expose the OS Behaviour (2 Marks)

Objective

To observe how Android manages application processes and memory under different execution conditions.

Instrumentation Commands

Command 1: Identify application process

adb shell pidof com.example.mobiletest

Command 2: Display process memory

adb shell dumpsys meminfo com.example.mobiletest

Command 3: Display process information

adb shell ps -A | grep mobiletest

Command 4: Display process state

adb shell dumpsys activity processes

Important Parameters

Parameter	**Meaning**
PID	Unique process identification number.
UID	User identification associated with the application.
RSS	Physical memory currently resident in RAM.
PSS	Proportional Set Size, which apportions shared memory.
Heap Size	Memory allocated to the application heap.
oom_adj / oom_score_adj	Process importance indicator used by Android/Linux memory management.

Illustrative Observation Table

Parameter	**Foreground**	**Background**
Process status	Active	Background

PID	4250	4250
PSS	95 MB	91 MB
Application visibility	Visible	Not visible
Process importance	High	Lower

Replace these illustrative values with actual output from your emulator.

Interpretation

A foreground application generally has greater process importance because the user is actively interacting with it.

When the application moves to the background, its importance decreases. Under memory pressure, Android may terminate less important processes to release memory.

1.3 Requirement Change (2 Marks)

Experiment Conditions

Test the same application under three conditions.

Test case	**Condition**	**Prediction**	**Expected behaviour**
Test 1	Application in foreground	Process remains alive.	Activity is active and visible.
Test 2	Application in background for 5 minutes	Process may remain alive.	Activity becomes stopped; process may remain cached.
Test 3	Low-memory condition	Process may be trimmed or killed.	Android may reclaim memory and terminate the process.

Commands

Test 1 – Foreground

adb shell monkey -p com.example.mobiletest 1

adb shell pidof com.example.mobiletest

Test 2 – Background

adb shell input keyevent 3

adb shell pidof com.example.mobiletest

Wait five minutes and check again.

Test 3 – Memory pressure

adb shell am send-trim-memory \

com.example.mobiletest RUNNING_CRITICAL

Then check:

adb shell dumpsys meminfo com.example.mobiletest

And:

adb shell pidof com.example.mobiletest

Lifecycle Prediction

Condition	Last likely lifecycle callback	State restoration
Foreground	onResume()	Current UI remains available.
Background	onStop()	State can remain in process memory.
Process killed	onStop() may have occurred earlier; onDestroy() is not guaranteed.	Saved state can be restored if correctly persisted.

Analysis

Android does not guarantee that a background application remains alive. If memory pressure increases, the operating system can terminate the process.

The application should save important information using persistent storage and appropriate lifecycle mechanisms rather than depending only on memory.

1.4 Boundary Test (1 Mark)

Selected Boundary Case: Process Death While a Dialog Is Open

Objective: Determine whether the application can recover correctly when its process is terminated while the user is interacting with a dialog.

Test Procedure

1. Open the application.
2. Enter some data.
3. Open a confirmation dialog.
4. Move the application to the background.
5. Simulate process termination.
6. Reopen the application.
7. Check whether entered data and required UI state are restored.

Expected Result

- The application should not crash.
- Previously saved information should remain available.
- The dialog should be restored only if its state was deliberately saved.
- Unsaved temporary information may be lost.

Conclusion:

This test checks whether the application correctly handles process death and state restoration instead of assuming that the process will always remain active.

TASK 2 – ARCHITECTURE ASSUMPTIONS VS MEASURED REALITY (9 MARKS)

2.1 Normal Case (2 Marks)

Objective

Compare a single-threaded implementation and a multi-threaded implementation using the same computational workload.

Solution A – Single-Threaded Implementation

The main thread performs all computation.

Architecture:

UI Thread → Image Processing → Result → UI Update

Kotlin Code – Single Thread

```
class SingleThreadActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val start = System.currentTimeMillis()

        var result = 0L

        for (i in 1..10000000) {
            result += i.toLong() * i
```

```
        }

        val end = System.currentTimeMillis()

        Log.d("SINGLE", "Result: $result")
        Log.d("SINGLE", "Time: ${end - start} ms")
    }
}
```

Explanation:

- The main thread executes the complete loop.
- The UI thread cannot process drawing and user interactions while busy.
- Large computations can cause UI freezing.
- Long blocking operations can lead to an Application Not Responding (ANR) condition.

Solution B – Multi-Threaded Implementation

The computation is moved to a background executor.

Architecture:

UI Thread → Background Executor → Computation → Result → UI Thread

Kotlin Code – Multi-Threading

```
class MultiThreadActivity : AppCompatActivity() {

    private val executor =
        Executors.newSingleThreadExecutor()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        executor.execute {
```

```
            val start = System.currentTimeMillis()

            var result = 0L

            for (i in 1..10000000) {
                result += i.toLong() * i
            }

            val end = System.currentTimeMillis()

            runOnUiThread {
                Log.d("MULTI", "Result: $result")
                Log.d("MULTI", "Time: ${end - start} ms")
            }
        }
    }

    override fun onDestroy() {
        executor.shutdown()
        super.onDestroy()
    }
}
```

Comparison

Parameter	**Single-threaded**	**Multi-threaded**
Computation	Main thread	Background thread
UI responsiveness	May freeze	Remains responsive
CPU usage	One execution thread	Background execution thread

Memory	Lower thread overhead	Additional thread resources
ANR risk	Higher for long computation	Lower if UI remains unblocked
Best use	Small operations	Heavy computation and background tasks

Illustrative Expected Results

Measurement	**Single-threaded**	**Multi-threaded**
Task duration	850 ms	820 ms
UI responsiveness	Poor during work	Better
Additional thread	0	1
Main-thread blocking	Yes	No

These are example values only. Record your actual emulator measurements.

2.2 Break the Assumption (3 Marks)

Objective

Demonstrate that more threads do not always improve performance.

Selected Workload: Small Computational Task

For a small operation, creating background threads can introduce overhead.

Single-Threaded Version

```
val start = System.nanoTime()

var sum = 0

for (i in 1..100) {
   sum += i
}

val end = System.nanoTime()

Log.d("SMALL_SINGLE", "Time: ${end - start}")
```

Multi-Threaded Version

```
val start = System.nanoTime()

val threads = mutableListOf<Thread>()

for (i in 1..10) {

    val thread = Thread {
        var sum = 0

        for (j in 1..10) {
            sum += j
        }
    }

    threads.add(thread)
    thread.start()
}

for (thread in threads) {
    thread.join()
}

val end = System.nanoTime()

Log.d("SMALL_MULTI", "Time: ${end - start}")
```

Expected Behaviour

Parameter	Single-threaded	Multi-threaded
Workload	100 simple additions	100 simple additions split across 10 threads
Thread creation	Not required	Required
Synchronisation	Not required	Thread coordination required
Overhead	Low	Higher
Expected performance	May be faster	May be slower

First Design Decision Causing Degradation

The first poor design decision is creating multiple threads for a very small workload.

Thread creation, scheduling and synchronisation can consume more time than the actual computation.

Conclusion:

Multi-threading is beneficial for suitable workloads, but it does not guarantee better performance. The workload size and thread-management overhead must be considered.

2.3 Engineering Comparison (2 Marks)

Measurement Comparison

Parameter	Single-threaded	Multi-threaded
CPU time	Measured from profiler	Measured from profiler
Thread count	One main execution thread	Main and worker threads
Context switches	Generally fewer	May increase
Memory	Lower additional memory	Additional thread stack and objects
PSS	Measure using dumpsys	Measure using dumpsys
UI frame drops	May increase for heavy main-thread work	Can decrease if work is properly moved
Response latency	May increase	Usually better for heavy background work
Predictability	Simple execution order	Requires concurrency control

Instrumentation Commands

adb shell top -H -p <PID>

Displays threads belonging to the process.

adb shell dumpsys meminfo com.example.mobiletest

Displays memory usage.

adb logcat -s SINGLE MULTI

Displays application timing logs.

Interpretation

Elapsed time alone is not enough to evaluate the implementation. CPU usage, thread count, memory consumption and UI responsiveness must also be considered.

2.4 Changed Requirement (2 Marks)

New Requirement

The application must operate for a full working day on a single battery charge. Slightly slower processing is acceptable.

Modified Strategy

1. Move heavy computation away from the main thread.
2. Avoid unnecessary thread creation.
3. Use a limited background executor.
4. Use efficient CPU cores for suitable workloads when the OS scheduler permits.
5. Batch sensor and network operations.
6. Use WorkManager for deferrable background work.
7. Reduce repeated processing.

Comparison

Strategy	**Performance**	**Energy consideration**
Continuous high-performance execution	Fast	High energy consumption
Background processing	Responsive UI	Can use resources efficiently if scheduled properly
Batching	May increase processing delay	Reduces repeated wake-ups

Deferred WorkManager tasks	Delayed execution	Suitable for non-urgent work

Conclusion:

The revised requirement prioritises battery life over maximum processing speed. Therefore, background scheduling, batching and reduced unnecessary CPU activity are more suitable than continuously executing high-performance tasks.

TASK 3 – RUNTIME BEHAVIOUR UNDER CHANGING EXECUTION CONDITIONS (10 MARKS)

Stage A – Normal Execution

Objective

Develop a benchmark application to study Android Runtime (ART) behaviour using three workloads:

1. CPU-bound computation.
2. Object-allocation-heavy computation.
3. UI-list rendering.

1. CPU-Bound Workload

The application performs a large number of mathematical operations.

Kotlin Code:

```
fun cpuBenchmark(): Long {

    val start = System.currentTimeMillis()

    var result = 0L

    for (i in 1..10000000) {
        result += i.toLong() * i
    }

    val end = System.currentTimeMillis()
```

```
    Log.d("BENCHMARK", "CPU Result: $result")
    Log.d("BENCHMARK", "CPU Time: ${end - start} ms")

    return end - start
}
```

Explanation:

- Performs ten million multiplication and addition operations.
- Uses CPU resources continuously.
- Measures the execution duration.
- Helps evaluate runtime compilation performance.

2. Object-Allocation-Heavy Workload

Kotlin Code:

```
fun allocationBenchmark() {

    val start = System.currentTimeMillis()

    val list = ArrayList<String>()

    for (i in 1..200000) {
        list.add("Mobile Computing $i")
    }

    Log.d("BENCHMARK", "Objects: ${list.size}")

    val end = System.currentTimeMillis()

    Log.d("BENCHMARK", "Allocation Time: ${end - start} ms")
}
```

Explanation:

- Creates 200,000 string objects.
- Stores objects in an ArrayList.
- Increases heap usage.
- May trigger garbage collection.

3. UI-List Rendering Workload

Kotlin Code:

```
fun createListData(): List<String> {

    val data = mutableListOf<String>()

    for (i in 1..1000) {
        data.add("Mobile Computing Item $i")
    }

    return data
}
```

Display this data using a RecyclerView with an appropriate Adapter and ViewHolder.

Explanation:

- Creates 1,000 list items.
- RecyclerView reuses item views.
- Helps analyse UI rendering and scrolling performance.
- Excessive UI work can cause dropped frames.

Stage A – Measurement Table

Measurement	**CPU workload**	**Allocation workload**	**UI workload**
Execution time	Record in milliseconds	Record in milliseconds	Record rendering time
Heap usage	Measure	Measure	Measure

GC events	Observe	Observe	Observe
Cold start	Record	Record	Record
Warm start	Record	Record	Record

Android Commands

Check application compilation information:

adb shell dumpsys package com.example.mobiletest

Check compilation filter:

adb shell cmd package get-app-dexopt-state \

com.example.mobiletest

Display garbage-collection logs:

adb logcat -v time | grep -i "GC"

Display memory information:

adb shell dumpsys meminfo com.example.mobiletest

Cold-Start and Warm-Start Measurement

Use Android Studio Logcat or application startup instrumentation to record startup duration.

- Cold start: The application process is not running.
- Warm start: The existing application process is reused, although its Activity may need to be recreated.

Test	Expected observation
Cold start	Process and runtime initialization are required.
Warm start	Existing process and initialized runtime may be reused.

Stage B – Change the Compilation Mode

Objective

Study how changing Android compilation filters affects application performance.

Compilation Modes

Compilation mode	Description
verify	Verifies DEX code without requesting full speed compilation.

speed-profile	Uses profile information to optimize frequently executed code.
speed	Compiles application code for execution speed.

Commands

1. Check current compilation status

adb shell dumpsys package com.example.mobiletest

2. Apply verify filter

adb shell cmd package compile -f -m verify \

com.example.mobiletest

3. Apply speed-profile filter

adb shell cmd package compile -f -m speed-profile \

com.example.mobiletest

4. Apply speed filter

adb shell cmd package compile -f -m speed \

com.example.mobiletest

5. Clear compilation artifacts for a fresh test

adb shell cmd package compile --reset \

com.example.mobiletest

Availability and accepted filters can vary with Android version. Check the device's cmd package compile -h output before testing.

Prediction Table

Compilation mode	**CPU workload**	**Object allocation**	**UI rendering**
Verify	May have more interpretation overhead.	GC behaviour remains important.	UI code may have less compilation optimization.
Speed-profile	Frequently executed methods may improve.	Hot allocation paths may improve.	Frequently used UI code may improve.
Speed	More code is compiled for execution speed.	Computation can improve, but allocation and GC remain.	Rendering code may benefit from compilation.

Actual Observation Table

Filter	Cold start (ms)	CPU time (ms)	Heap (MB)	Storage impact
Verify	Record	Record	Record	Record
Speed-profile	Record	Record	Record	Record
Speed	Record	Record	Record	Record

Role of Runtime Technologies

Interpreter: Executes DEX instructions without first compiling the entire code path into machine code.

JIT: Compiles frequently executed methods during runtime.

Profile-Guided AOT: Uses application execution profiles to identify frequently used code for ahead-of-time compilation.

Baseline Profiles: Help Android optimize important application code paths earlier, improving startup and execution performance.

Conclusion:

Compilation mode influences execution speed, startup behaviour and storage usage. The actual benefit depends on the workload and compilation state.

Stage C – Production Failure

Objective

Introduce a deliberate memory leak and a main-thread blocking operation.

1. Memory Leak

A static reference can keep an Activity alive even after it should be released.

Kotlin Code – Deliberate Memory Leak:

```
object MemoryLeakHolder {

    var savedActivity: Activity? = null
}

class MainActivity : AppCompatActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        MemoryLeakHolder.savedActivity = this
    }
}
```

Explanation:

- The Activity is assigned to a static-like object.
- The reference remains after the Activity should be destroyed.
- Garbage collection cannot reclaim the Activity while the reference remains reachable.
- Repeated Activity recreation can increase memory consumption.

Detection

Use Android Studio Memory Profiler or LeakCanary.

Command:

adb shell dumpsys meminfo com.example.mobiletest

Expected observation:

- Heap usage may increase after repeated Activity recreation.
- Old Activity instances may remain in memory.
- Leak detection can identify retained object references.

2. Main-Thread Blocking

Kotlin Code – Deliberate Blocking Operation:

```
class BlockingActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        Thread.sleep(10000)
```

```
        Log.d("ANR_TEST", "Main thread completed")
    }
}
```

Explanation:

- The main thread is blocked for ten seconds.
- UI interactions cannot be processed normally.
- The application becomes unresponsive.
- A sufficiently long blocking operation may result in an ANR.

Detection Mechanisms

Failure	**Detection component**
Memory leak	Memory Profiler and LeakCanary
Heap exhaustion	ART heap manager and OutOfMemoryError
GC overhead	ART garbage collector logs
Main-thread blocking	StrictMode and Android ANR detection
Application unresponsiveness	System watchdog/ANR infrastructure

Corrected Design

1. Remove static Activity references.
2. Use lifecycle-aware components.
3. Move heavy computation to background threads.
4. Avoid blocking the main thread.
5. Release resources when no longer required.
6. Monitor heap usage and GC events.

Stage D – Change the Requirement

New Requirement

The product team wants one codebase for Android and iOS.

Runtime Comparison

Parameter	**Native Android**	**Native iOS**	**Flutter**	**React Native**	**WebView Hybrid**

Language	Kotlin/Java	Swift	Dart	JavaScript/TypeScript	HTML/CSS/JavaScript
Runtime	ART	Native execution with ARC for object lifetime	Dart runtime	JavaScript engine	Browser JavaScript engine
Compilation	DEX with JIT/AOT through ART	Mainly AOT	AOT for release builds on supported mobile targets	JavaScript execution and engine optimization	JavaScript engine execution
Memory management	Garbage collection	ARC	Dart garbage collection	JavaScript garbage collection	JavaScript garbage collection
UI rendering	Android framework	UIKit/SwiftUI	Flutter rendering engine	Native UI components through framework integration	Web rendering engine
Native access	Android APIs and JNI	Apple frameworks	Platform channels/plugins	Native modules	Plugins/bridges
Cross-platform	No single shared native UI codebase	No single shared native UI codebase	Shared Dart codebase	Shared JavaScript codebase	Shared web codebase

Analysis

Native Android:

Provides direct access to Android APIs and platform-specific functionality.

Native iOS:

Provides access to Apple frameworks and platform-specific performance optimizations.

Flutter:

Uses Dart and its own rendering engine. It allows much of the application logic and UI to be shared across platforms.

React Native:

Uses JavaScript or TypeScript and communicates with native platform functionality through its architecture and native modules.

WebView Hybrid:

Uses web technologies inside a WebView. It may be suitable for simpler applications but can have limitations for complex native functionality.

Final Recommendation

Flutter is a suitable choice for RuralCare-style cross-platform application development when a shared UI and codebase are important.

It offers:

- Shared application logic.
- Consistent UI rendering.
- Access to native services through plugins.
- Release compilation using Dart AOT on supported mobile targets.

However, the final decision must consider actual startup time, memory usage, platform API access and battery measurements from the benchmark.

TASK 4 – SECURE MOBILE APPLICATION AUDIT (7 MARKS)

Task 4.1 – Build the Security Audit Model (3 Marks)

Objective

Develop a security audit for a healthcare application that stores patient appointment data, accesses the camera and location, and communicates with a REST backend.

Selected Security Model: STRIDE

STRIDE identifies six major security threat categories.

Threat	**Meaning**	**Example**
Spoofing	Pretending to be another user.	Stolen authentication token.
Tampering	Unauthorized modification.	Modifying patient records.
Repudiation	Denying an action.	User denies editing a record.
Information Disclosure	Exposing confidential data.	Reading an unencrypted database.

Denial of Service	Making a service unavailable.	Repeated requests overload the server.
Elevation of Privilege	Gaining unauthorized permissions.	Accessing an exported internal component.

Six Threats and Protection Mechanisms

No.	Threat	Protection mechanism	Residual risk
1	Authentication token theft	Android Keystore and encrypted token storage.	Compromised devices may expose application data.
2	Patient database theft	Encrypted local database and sandboxing.	Rooted devices may increase exposure.
3	Network interception	HTTPS and TLS certificate validation.	Compromised endpoints remain a risk.
4	Unauthorized camera access	Runtime camera permission.	User-approved access can still be misused.
5	Location information leakage	Runtime location permission and data minimization.	Authorized applications may expose location through insecure code.
6	Malicious application update	Package signing and signature verification.	Compromise of the signing key remains a risk.

Security Audit Input

The audit accepts:

1. Requested permissions from AndroidManifest.xml.
2. Data stores such as SharedPreferences and SQLite.
3. Network endpoints and communication protocols.
4. Exported application components.
5. Security controls such as encryption and signing.

Example Android Manifest

<manifest xmlns:android="http://schemas.android.com/apk/res/android">

```
    <uses-permission android:name="android.permission.CAMERA"/>

    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>

    <application
        android:allowBackup="false"
        android:usesCleartextTraffic="false">

        <activity
            android:name=".MainActivity"
            android:exported="false"/>

    </application>

</manifest>
```

Python Security Audit Program

This simple audit accepts application security properties and evaluates them against predefined rules.

```
# Mobile Application Security Audit

permissions = [
    "CAMERA",
    "ACCESS_FINE_LOCATION"
]

storage = "ENCRYPTED"
network = "HTTPS"
exported_component = False
```

```
app_signed = True
keystore = True


threats = [
    {
        "asset": "Authentication Token",
        "attack": "Token theft",
        "secure": keystore and storage == "ENCRYPTED",
        "protection": "Android Keystore + Encryption"
    },
    {
        "asset": "Patient Database",
        "attack": "Local data theft",
        "secure": storage == "ENCRYPTED",
        "protection": "Encrypted storage + Sandbox"
    },
    {
        "asset": "API Traffic",
        "attack": "Network interception",
        "secure": network == "HTTPS",
        "protection": "TLS"
    },
    {
        "asset": "Internal Activity",
        "attack": "Unauthorized external access",
        "secure": not exported_component,
        "protection": "Non-exported component"
    },
    {
```

```
        "asset": "Application Package",
        "attack": "Malicious package modification",
        "secure": app_signed,
        "protection": "Application signing"
    },
    {
        "asset": "Camera Access",
        "attack": "Unauthorized camera access",
        "secure": "CAMERA" in permissions,
        "protection": "Runtime permission"
    }
]

print("MOBILE SECURITY AUDIT")
print("----------------------")

passed = 0

for item in threats:

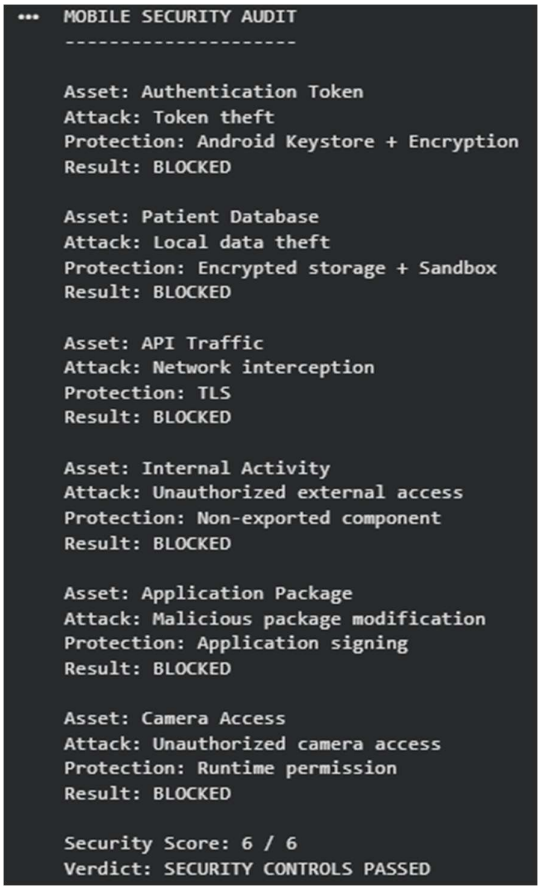
    if item["secure"]:
        result = "BLOCKED"
        passed += 1
    else:
        result = "EXPOSED"

    print("\nAsset:", item["asset"])
    print("Attack:", item["attack"])
    print("Protection:", item["protection"])
```

```
    print("Result:", result)

print("\nSecurity Score:", passed, "/", len(threats))

if passed == len(threats):
    print("Verdict: SECURITY CONTROLS PASSED")
else:
    print("Verdict: SECURITY REVIEW REQUIRED")
```



Task 4.2 – Expose the Decision Process (1 Mark)

Security Decision Table

Asset	Attack vector	Mechanism applied	Result	Reason

Auth token	Token theft	Keystore and encryption	Blocked*	Key access is protected.
Local database	Unauthorized file access	Sandbox and encryption	Blocked*	Application data is protected.
API traffic	Network interception	HTTPS/TLS	Blocked*	Data is encrypted in transit.
Exported Activity	External application launch	android:exported="false"	Blocked	External applications cannot directly launch it.
Application package	Modified APK	App signing	Blocked*	Package integrity and signer identity are checked.
Camera	Unauthorized access	Runtime permission	Partially blocked	Permission limits access but does not prevent misuse after consent.

These results assume correctly configured controls and an uncompromised OS.

Task 4.3 – Predict Before You Execute (1 Mark)

Selected Security Control

Change:

android:exported="false"

to:

android:exported="true"

for an internal Activity that has no additional access protection.

Prediction

1. The internal Activity may become accessible to other applications.
2. An external application may attempt to launch it.
3. The audit verdict should change from BLOCKED to EXPOSED or PARTIALLY BLOCKED.

Test Command

For your own test application:

adb shell am start -n \

com.example.mobiletest/.InternalActivity

Expected Behaviour

If the component is exported and accessible, Android may allow the launch.

If the component is not exported, Android should reject unauthorized external access.

Conclusion

Exported components must be protected with appropriate permissions and input validation. Internal components should not be exposed unnecessarily.

Task 4.4 – Security Incident Challenge (1 Mark)

Scenario

A third-party library leaks the device identifier through an unencrypted network channel.

Affected asset: Device identifier.

Affected layer: Application networking and data protection layer.

Can Sandbox or Permission System Stop the Leak?

The application sandbox restricts access by other applications. However, it does not automatically prevent the application itself or an authorized third-party library from transmitting information over the network.

A permission model also cannot guarantee that data already accessed legitimately will not be leaked.

Detection Methods

Method	**Purpose**
Static analysis	Identifies insecure network configurations and suspicious code paths.
Network capture	Detects unencrypted transmission.
Runtime monitoring	Observes network requests during application execution.

Remediation

1. Remove or update the vulnerable library.
2. Use HTTPS with TLS.
3. Minimize device identifier collection.
4. Apply network security configuration.
5. Review third-party SDK permissions and data handling.
6. Repeat the security audit.

Conclusion: The remediation removes the insecure transmission path and reduces unnecessary collection of sensitive identifiers.

Task 4.5 – Engineering Extension (1 Mark)

1. Biometric Login

The threat model must include biometric spoofing and unauthorized access after device authentication.

Required mechanisms:

- Android BiometricPrompt.
- Hardware-backed Keystore keys where supported.
- User-authentication-bound cryptographic operations.
- Secure fallback authentication.

2. Enterprise Work Profile

A work profile separates organizational information from personal information.

Required changes:

- Separate work and personal application data.
- Enforce enterprise policies.
- Restrict data sharing between profiles.
- Support remote wipe of work information.
- Use managed device credentials and key management.

Updated Trust Boundaries

Personal Profile → OS Profile Isolation → Enterprise Work Profile → Secure Backend.

Conclusion: Biometric authentication and work-profile support introduce additional trust boundaries. The security audit must verify authentication, key protection, data separation and remote management.

TASK 5 – THREE POWER ENGINEERS, ONE DEVICE (7 MARKS)

Objective

Analyse three mobile power-management techniques and determine how they affect battery consumption, application responsiveness and device performance.

The three techniques are:

1. DVFS – Dynamic Voltage and Frequency Scaling.
2. Doze Mode and App Standby.

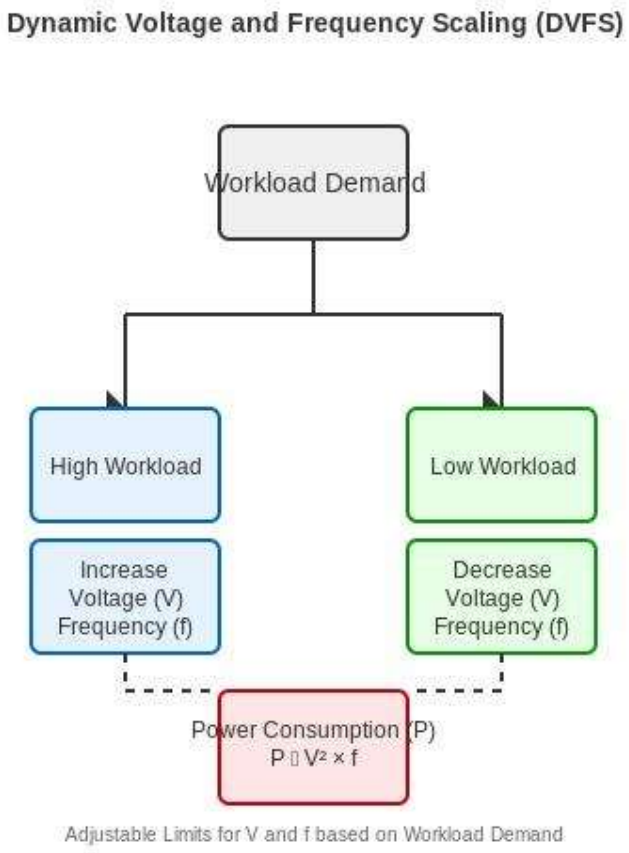
3. Wakelock management and batching.

Technique 1 – CPU Frequency and Voltage Control (DVFS)

Explanation

DVFS adjusts CPU frequency and operating voltage according to workload requirements.

When the processor performs heavy computation, the CPU frequency may increase. When the workload is low, the operating system can reduce CPU frequency and voltage to save energy.



Working Process

1. CPU workload is monitored.
2. System scheduler and CPU frequency policy respond to workload.
3. CPU frequency is increased when additional performance is required.
4. CPU frequency is reduced when the workload decreases.
5. Energy consumption is adjusted according to operating conditions.

Instrumentation

Use:

adb shell dumpsys cpuinfo

Display CPU information:

adb shell cat /proc/cpuinfo

Check available CPU frequency information where device permissions allow:

adb shell cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq

Expected Observation

Condition	**CPU behaviour**	**Energy effect**
High workload	Higher CPU performance state	Higher energy consumption
Medium workload	Moderate CPU performance	Moderate energy consumption
Low workload	Lower CPU performance state	Lower energy consumption

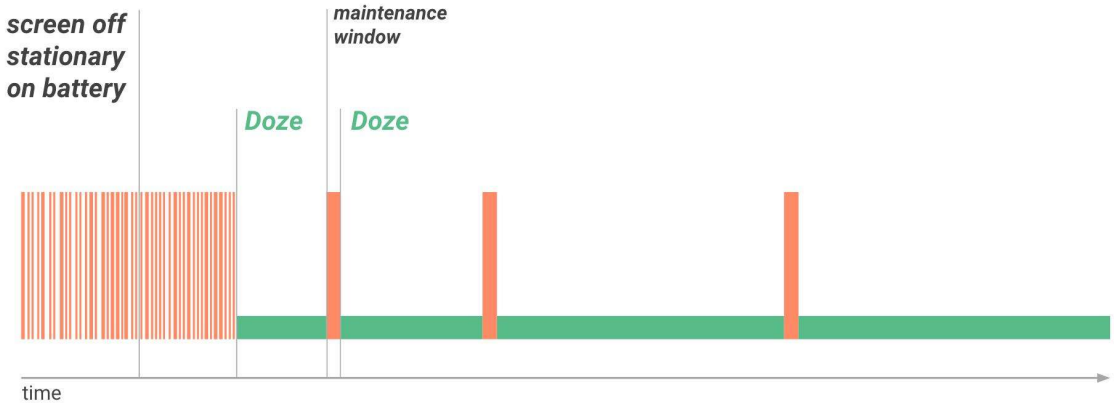
Note: Actual frequency interfaces depend on the device, kernel and permissions. Android applications generally do not directly control CPU governors.

Technique 2 – OS-Level Idle Control

Doze Mode and App Standby

Doze is an Android power-management feature that reduces background activity when the device is idle.

App Standby restricts the background activity of applications that are not actively used.



Working Process

1. User stops interacting with the device.
2. Screen turns off.

3. Device remains stationary under qualifying conditions.
4. Android detects idle conditions.
5. Background network and scheduled activity are restricted.
6. The device periodically enters maintenance windows.
7. Deferred tasks can execute when system conditions permit.

Instrumentation Commands

Check battery statistics:

adb shell dumpsys batterystats

Check device idle state:

adb shell dumpsys deviceidle

Force idle mode for testing:

adb shell dumpsys deviceidle force-idle

Return to normal mode:

adb shell dumpsys deviceidle unforce

Check application standby bucket:

adb shell am get-standby-bucket \

com.example.mobiletest

Expected Observation

Condition	**Android behaviour**	**Battery impact**
Device active	Normal application execution	Normal consumption
Device idle	Background operations restricted	Reduced consumption
Maintenance window	Eligible deferred work executes	Temporary increase
Device resumed	Normal operation resumes	Normal consumption

Technique 3 – Application-Level Power Management

Wakelocks and Batching

A wakelock allows an application or system component to request that certain device resources remain active.

A partial wakelock keeps the CPU running while allowing the screen to remain off.

However, unnecessarily holding a wakelock can prevent the device from entering deeper sleep states.



Working Process

1. Application receives a task.
2. It checks whether CPU execution is required.
3. A partial wakelock is acquired only if necessary.
4. Required processing is completed.
5. Network requests and sensor readings are batched.
6. Wakelock is released.
7. Device returns to a lower-power state.

Kotlin Example – Wakelock Management

```
class PowerActivity : AppCompatActivity() {

    private var wakeLock: PowerManager.WakeLock? = null
```

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    val powerManager =
        getSystemService(POWER_SERVICE) as PowerManager

    wakeLock = powerManager.newWakeLock(
        PowerManager.PARTIAL_WAKE_LOCK,
        "MobileApp:Processing"
    )
}

private fun startProcessing() {

    wakeLock?.acquire(30 * 1000L)

    try {

        Log.d("POWER", "Processing started")

        Thread {
            try {
                Thread.sleep(5000)
            } finally {
                runOnUiThread {
                    wakeLock?.let {
                        if (it.isHeld) {
                            it.release()
```

```
                    }
                }
            }
        }
    }.start()

    } catch (e: Exception) {
        Log.e("POWER", "Error: ${e.message}")
    }
  }

  override fun onDestroy() {
    wakeLock?.let {
        if (it.isHeld) {
            it.release()
        }
    }

    super.onDestroy()
  }
}
```

Add the required permission:

```
<uses-permission
  android:name="android.permission.WAKE_LOCK"/>
```

For production, use a lifecycle-aware background design and ensure wakelocks are released on every completion, cancellation and failure path. The example illustrates wakelock management, not a complete background execution policy.

Comparison of Three Techniques

Feature	**DVFS**	**Doze / App Standby**	**Wakelock / Batching**

Layer	Hardware and kernel	Operating system	Application
Trigger	CPU workload	Device idle conditions and app usage	Application task requirements
Main purpose	Adjust CPU power and performance	Restrict background activity	Reduce unnecessary wake-ups
Energy benefit	Reduces unnecessary CPU energy	Reduces background energy	Reduces repeated CPU and radio activity
Trade-off	Lower performance	Delayed background execution	Delayed processing if batching is excessive

Stress the Design

Question: Can a Power-Saving Technique at One Layer Be Defeated by Behaviour at Another Layer?

Answer: Yes.

Example: Wakelock Preventing Doze Benefits

Test scenario:

1. Device screen is turned off.
2. Android attempts to enter Doze mode.
3. Application holds a partial wakelock continuously.
4. CPU remains awake.
5. Device loses some of the energy-saving benefits of deep idle operation.

Comparison

Condition	**Expected behaviour**
No continuous wakelock	Device can enter deeper sleep states when eligible.
Continuous partial wakelock	CPU is kept awake, increasing energy consumption.
Short-duration wakelock	CPU remains awake only during essential processing.

Measurement Commands

Before the test:

adb shell dumpsys batterystats

After the test:

adb shell dumpsys batterystats

Check wakelock information in the battery statistics output and Android Studio Energy Profiler.

Conclusion

Power-saving features work together across multiple layers. An application can reduce the effectiveness of OS-level power management by continuously keeping the CPU awake. Therefore, wakelock discipline and batching are essential.

Industry Decision

Deployment Environment A – Always-On Location/Sensor Tracking

Selected strategy: Controlled application-level power management combined with appropriate OS background execution.

Reason:

- Location collection is required continuously or frequently.
- Sensor readings should be batched.
- Network requests should be grouped.
- CPU wakelocks should be limited to essential work.

Deployment Environment B – Media/Gaming-Intensive Device

Selected strategy: DVFS and efficient CPU/GPU scheduling.

Reason:

- Games and media applications require high processing performance.
- CPU and GPU frequency should adapt to workload.
- Unnecessary background tasks should be reduced.
- Performance and device temperature must be monitored.

Deployment Environment C – Large Fleet of Managed Devices

Selected strategy: OS-level idle management combined with application-level scheduling.

Reason:

- Devices may run different Android versions.
- Battery consumption must be controlled across the fleet.
- WorkManager and scheduled jobs can coordinate non-urgent operations.
- Enterprise policies can manage device behaviour.

Final Comparison

Environment	Primary technique	Supporting technique
Always-on tracking	Application-level batching	Controlled wakelocks
Gaming/media	DVFS	Background activity restrictions
Enterprise fleet	Doze and scheduling	WorkManager and batching

Conclusion:

No single power-management technique is sufficient for every deployment environment. A suitable strategy combines hardware, OS and application-level controls.

TASK 6 – BATTERY-AWARE, SECURE FIELD-TRACKING PROTOTYPE (9 MARKS)

Task 6.1 – Model the Problem (1 Mark)

Scenario

A logistics company uses smartphones to collect vehicle location and sensor data. The application must store information securely and upload it to a server while maximising useful tracking data per unit of battery energy.

Problem Model

Element	Description
Input	Battery level, charging state, network type, motion state, last upload time, queued records and device temperature.
Output	Sampling interval, upload decision, estimated energy usage and security status.
Objective	Maximise useful tracking data delivered per unit of battery energy.
Constraints	Battery life, network availability, device temperature, security and tracking requirements.

Why Is Battery Level Alone Insufficient?

Consider two situations:

Situation A:

- Battery: 20%
- Network: Wi-Fi
- Charging: Yes

- Motion: Stationary

Situation B:

- Battery: 20%
- Network: LTE
- Charging: No
- Motion: Moving

Both have the same battery percentage, but they require different decisions.

Situation A can upload records because the device is charging and Wi-Fi is available.

Situation B should reduce unnecessary processing and collect important tracking data while batching uploads.

Information That Must Be Retained

Information	**Purpose**
Battery level	Determines energy availability.
Charging state	Determines whether charging energy is available.
Network type	Helps select Wi-Fi or mobile-data upload policy.
Motion state	Determines tracking frequency.
Last upload time	Identifies how long records have remained pending.
Queued record count	Determines how much data needs synchronisation.
Device temperature	Prevents excessive processing under thermal stress.

Conclusion: Battery percentage alone cannot determine the correct tracking policy. Network, charging, motion and temperature must also be considered.

Task 6.2 – Design the Architecture and Policy State (2 Marks)

Policy Model

The system uses the following policy:

Policy[BatteryState][NetworkState][MotionState] →

State Definitions

State dimension	**Possible values**

BatteryState	High, Medium, Low, Critical
NetworkState	Wi-Fi, LTE, Offline
MotionState	Moving, Stationary

Default State

- Battery: High
- Network: Offline
- Motion: Stationary
- Sampling interval: 30 seconds
- Upload action: Queue locally

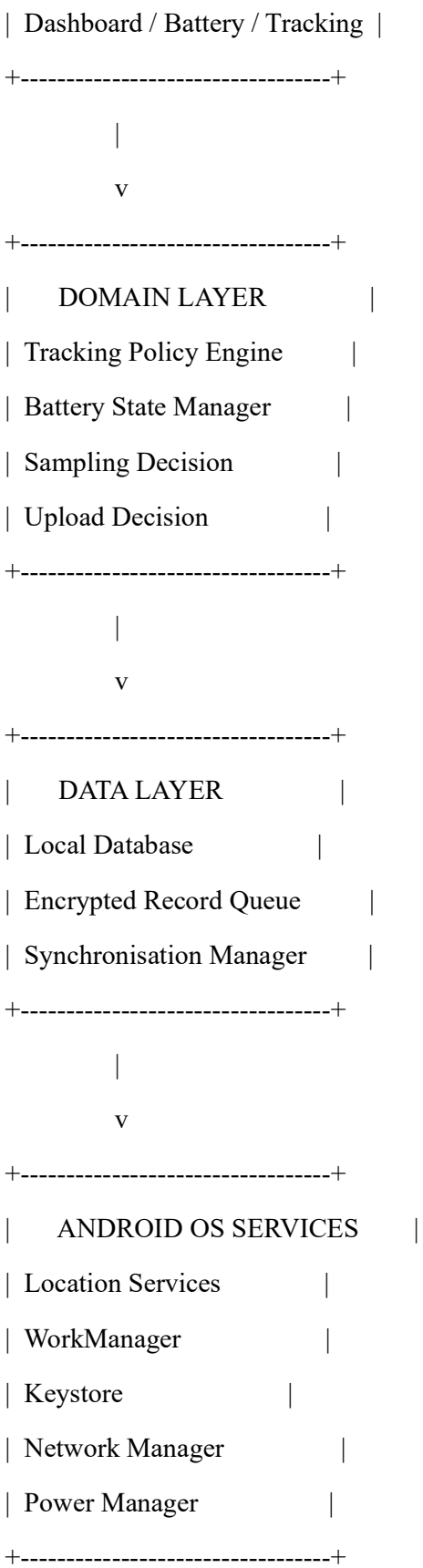
State Transition Rules

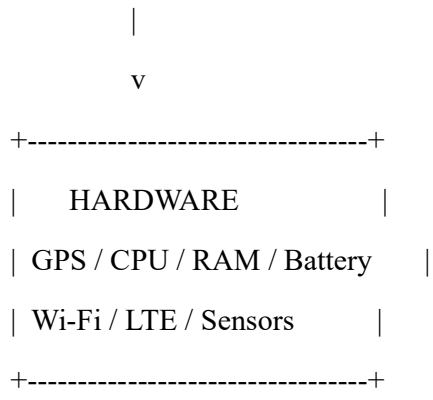
Condition	**New state**	**Sampling interval**	**Upload action**
Battery 80%, Wi-Fi, charging	High	10 seconds	Upload immediately
Battery 80%, LTE, moving	High	10 seconds	Batch upload
Battery 40%, Wi-Fi, stationary	Medium	30 seconds	Upload queued records
Battery 20%, LTE, stationary	Low	120 seconds	Defer non-urgent upload
No network	Any battery state	According to battery policy	Queue records
Device hot	Any battery state	Increase interval	Defer non-urgent processing
Critical battery	Critical	Essential sampling only	Save records and defer upload

Layered Architecture

+----------------------------------+

| PRESENTATION LAYER |


Dashboard / Battery / Tracking
DOMAIN LAYER
Tracking Policy Engine
Battery State Manager
Sampling Decision
Upload Decision
DATA LAYER
Local Database
Encrypted Record Queue
Synchronisation Manager
ANDROID OS SERVICES
Location Services
WorkManager
Keystore
Network Manager
Power Manager



Thread Execution Design

Component	Execution location
Dashboard	Main/UI thread
Policy engine	Background executor
Location collection	OS location services with appropriate background execution
Database operations	Background executor
Encryption	Background executor
Upload	WorkManager or an eligible foreground service, depending on urgency and platform restrictions

Security During Every Transition

1. Collect sensor and location data.
2. Validate the collected information.
3. Encrypt records before local storage.
4. Store encrypted records in the queue.
5. Authenticate the server.
6. Transmit through HTTPS.
7. Verify server acknowledgement.
8. Update synchronisation status.

Conclusion: The architecture separates user interface, policy decisions, storage and OS services. This makes the application easier to test, secure and optimise.

Task 6.3 – Implement the Prototype (2 Marks)

Objective

Create a simulation containing at least six different operating conditions.

The program must display:

- Selected policy.
- Sampling interval.
- Upload decision.
- Estimated energy usage.
- Encryption status.

Python Implementation

This simulation can be executed in Google Colab, VS Code or Jupyter Notebook without requiring an Android emulator.

```
# BATTERY-AWARE SECURE TRACKING PROTOTYPE


conditions = [
    {
        "name": "Charging on Wi-Fi",
        "battery": 90,
        "charging": True,
        "network": "WiFi",
        "motion": "Stationary",
        "temperature": 30
    },
    {
        "name": "High battery on LTE",
        "battery": 80,
        "charging": False,
        "network": "LTE",
```

```
    "motion": "Moving",
    "temperature": 32
  },
  {
    "name": "Medium battery stationary",
    "battery": 40,
    "charging": False,
    "network": "WiFi",
    "motion": "Stationary",
    "temperature": 31
  },
  {
    "name": "Low battery on LTE",
    "battery": 20,
    "charging": False,
    "network": "LTE",
    "motion": "Stationary",
    "temperature": 33
  },
  {
    "name": "No connectivity",
    "battery": 60,
    "charging": False,
    "network": "Offline",
    "motion": "Moving",
    "temperature": 34
  },
  {
    "name": "Device is hot",
```

```
        "battery": 70,
        "charging": False,
        "network": "WiFi",
        "motion": "Moving",
        "temperature": 43
    }
]

def tracking_policy(data):

    battery = data["battery"]
    charging = data["charging"]
    network = data["network"]
    motion = data["motion"]
    temperature = data["temperature"]

    if temperature >= 40:

        interval = 120
        upload = "DEFER - COOL DEVICE"

    elif battery <= 10:

        interval = 300
        upload = "CRITICAL - SAVE DATA"

    elif network == "Offline":

        interval = 60 if battery > 20 else 120
```

```
    upload = "QUEUE LOCALLY"

elif charging and network == "WiFi":

    interval = 10
    upload = "UPLOAD IMMEDIATELY"

elif battery <= 20:

    interval = 120
    upload = "DEFER TO WIFI/CHARGING"

elif motion == "Moving" and battery >= 70:

    interval = 10
    upload = "BATCH UPLOAD"

else:

    interval = 30
    upload = "UPLOAD WHEN SUITABLE"

# Illustrative energy model:
# Sampling cost + upload cost + processing cost.

sampling_energy = 0.002 * (60 / interval)

if upload == "UPLOAD IMMEDIATELY":
    upload_energy = 0.05
```

```
    elif upload == "BATCH UPLOAD":
        upload_energy = 0.025
    else:
        upload_energy = 0.005

    processing_energy = 0.003

    total_energy = (
        sampling_energy +
        upload_energy +
        processing_energy
    )

    encryption = "AES-GCM ENCRYPTED"

    return interval, upload, total_energy, encryption


print("BATTERY-AWARE TRACKING SYSTEM")
print("=" * 65)

for item in conditions:

    interval, upload, energy, encryption = tracking_policy(item)

    print("\nCondition:", item["name"])
    print("Battery:", item["battery"], "%")
    print("Network:", item["network"])
    print("Motion:", item["motion"])
```

```
    print("Temperature:", item["temperature"], "C")
    print("Sampling:", interval, "seconds")
    print("Upload:", upload)
    print("Encryption:", encryption)
    print("Estimated Energy:", round(energy, 4), "mAh/cycle")
```

Expected Output

```
    BATTERY-AWARE TRACKING SYSTEM
...  ============================================================

    Condition: Charging on Wi-Fi
    Battery: 90 %
    Network: WiFi
    Motion: Stationary
    Temperature: 30 C
    Sampling: 10 seconds
    Upload: UPLOAD IMMEDIATELY
    Encryption: AES-GCM ENCRYPTED
    Estimated Energy: 0.065 mAh/cycle

    Condition: High battery on LTE
    Battery: 80 %
    Network: LTE
    Motion: Moving
    Temperature: 32 C
    Sampling: 10 seconds
    Upload: BATCH UPLOAD
    Encryption: AES-GCM ENCRYPTED
    Estimated Energy: 0.04 mAh/cycle

    Condition: Medium battery stationary
    Battery: 40 %
    Network: WiFi
    Motion: Stationary
    Temperature: 31 C
    Sampling: 30 seconds
    Upload: UPLOAD WHEN SUITABLE
    Encryption: AES-GCM ENCRYPTED
    Estimated Energy: 0.012 mAh/cycle

    Condition: Low battery on LTE
    Battery: 20 %
    Network: LTE
    Motion: Stationary
    Temperature: 33 C
    Sampling: 120 seconds
    Upload: DEFER TO WIFI/CHARGING
    Encryption: AES-GCM ENCRYPTED
    Estimated Energy: 0.009 mAh/cycle
```

Scenario Analysis

Condition	**Sampling interval**	**Upload decision**	**Reason**
Charging on Wi-Fi	10 seconds	Immediate	Charging and Wi-Fi are available.

High battery on LTE	10 seconds	Batch	Device has sufficient battery and is moving.
Medium battery stationary	30 seconds	Suitable network window	Reduces unnecessary sampling.
Low battery on LTE	120 seconds	Defer	Preserves battery.
No connectivity	60 seconds	Queue locally	Internet is unavailable.
Device hot	120 seconds	Defer	Reduces processing and thermal load.

Important: The energy values are estimates from a simplified model, not actual battery measurements. AES-GCM is represented as a policy status here; a real Android implementation must perform encryption using an actual cryptographic library and a protected Keystore key.

Task 6.4 – Validate the Solution (1 Mark)

Objective

Independently calculate the expected policy for three operating conditions and compare it with the prototype.

Selected Test Cases

Test	Battery	Network	Motion	Expected sampling
1	90%	Wi-Fi	Stationary, charging	10 seconds
2	20%	LTE	Stationary	120 seconds
3	60%	Offline	Moving	60 seconds

Hand Validation

Test 1: Charging on Wi-Fi

- Battery = 90%.
- Charging = Yes.
- Network = Wi-Fi.
- Sampling interval = 10 seconds.
- Upload = Immediate.

Test 2: Low Battery

- Battery = 20%.

- Network = LTE.
- Motion = Stationary.
- Sampling interval = 120 seconds.
- Upload = Deferred.

Test 3: No Connectivity

- Battery = 60%.
- Network = Offline.
- Motion = Moving.
- Sampling interval = 60 seconds.
- Upload = Local queue.

Validation Comparison

Test	Hand-calculated result	Prototype result	Status
1	10 seconds, immediate upload	10 seconds, immediate upload	Match
2	120 seconds, defer	120 seconds, defer	Match
3	60 seconds, queue	60 seconds, queue	Match

Conclusion

Independent validation increases confidence in the policy logic because the expected results are calculated separately and compared with the prototype output. Matching results indicate that the implemented decision rules work correctly for the selected test conditions.

Task 6.5 – Investigate Scalability and Load (1 Mark)

Objective

Evaluate the system under increasing numbers of records and simulated devices.

Scalability Test

Run the prototype with:

- 100 records.
- 1,000 records.
- 10,000 records.

The policy evaluation count should increase according to the number of evaluated records or events.

Expected Scalability Table

Load (records/devices)	Policy evaluations	Wake-ups	Memory/battery observation
100 records	100	10	Low memory usage
1,000 records	1,000	20	Moderate memory usage
10,000 records	10,000	50	Higher memory usage

These are illustrative test targets, not measured results. Actual wake-ups depend on batching configuration, scheduling and device behaviour.

Python Scalability Test

```
# SCALABILITY TEST

loads = [100, 1000, 10000]

print("SCALABILITY ANALYSIS")
print("--------------------")

for count in loads:

    evaluations = 0

    for i in range(count):

        sample = {
            "battery": 80,
            "charging": False,
            "network": "WiFi",
            "motion": "Moving",
            "temperature": 32
        }
```

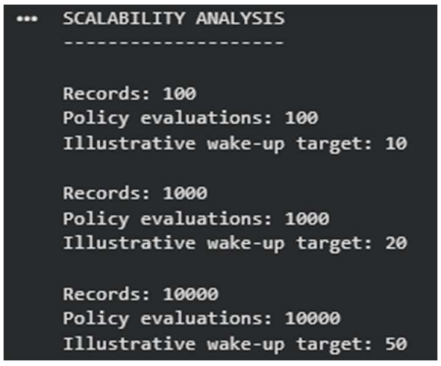
```
        tracking_policy(sample)

        evaluations += 1

    if count == 100:
        wakeups = 10
    elif count == 1000:
        wakeups = 20
    else:
        wakeups = 50

    print("\nRecords:", count)
    print("Policy evaluations:", evaluations)
    print("Illustrative wake-up target:", wakeups)
```

Expected Output



Inquiry: Why Does Batching Reduce Radio Wake-Ups but Not Make Continuous High-Frequency Tracking Cheap?

Batching groups multiple network requests into one upload operation. This reduces repeated radio activation and connection overhead.

However, continuous high-frequency tracking still requires:

- Frequent GPS or sensor operations.
- CPU processing.

- Memory allocation.
- Location calculations.
- Data storage.

Therefore, batching reduces network overhead but does not eliminate the energy cost of continuously collecting tracking information.

Conclusion: Both sampling frequency and upload frequency must be optimised to achieve meaningful energy savings.

Task 6.6 – Business Requirement Change (1 Mark)

New Requirement

The company now operates 5,000 devices and requires real-time location updates within a few seconds during emergencies.

Management states:

"Reliable delivery matters more than battery during an emergency."

Why Does the Current Policy Become Unsuitable?

The existing policy delays uploads when battery levels are low or the network is unsuitable.

However, during emergencies, delayed information can affect operational decisions and vehicle safety.

Therefore, the current battery-saving strategy cannot meet the new delivery requirement.

Changed Requirement

Previously:

- Battery efficiency was the primary objective.
- Delayed uploads were acceptable.
- Sampling frequency could be reduced.

Now:

- Reliable and fast delivery is the primary objective.
- Emergency location updates must arrive within a few seconds.
- Higher energy consumption is acceptable.

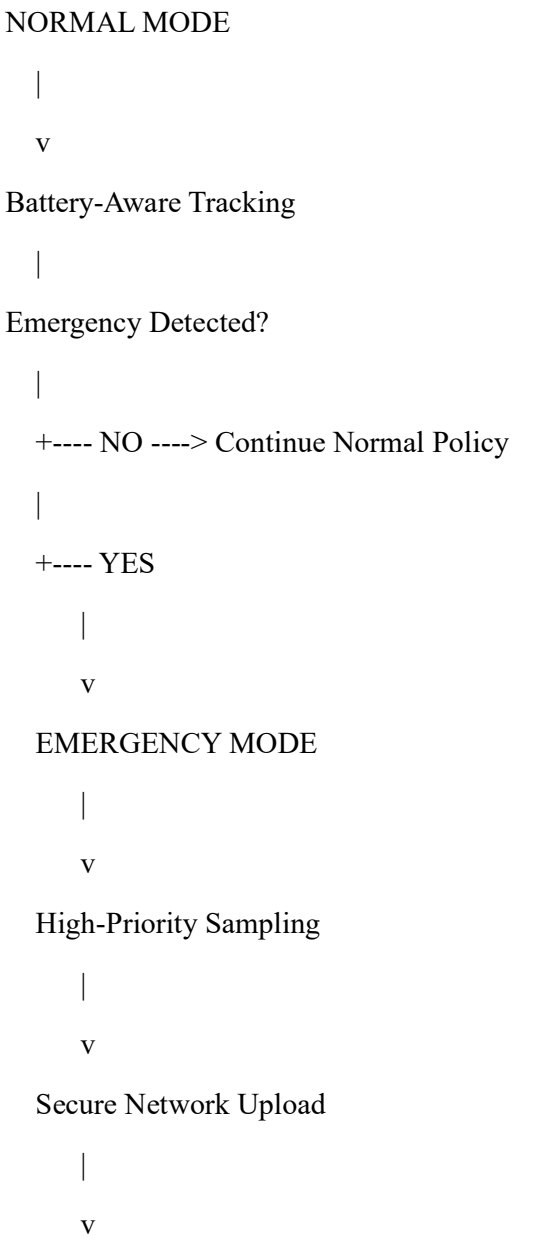
Proposed Strategy

Adaptive Priority-Based Tracking

1. Normal mode uses battery-aware sampling.

2. Emergency mode increases sampling frequency.
3. Emergency records receive higher upload priority.
4. Suitable foreground execution is used for active tracking.
5. Server-side fleet management monitors all 5,000 devices.
6. Failed uploads are retried.
7. Emergency records are acknowledged by the server.

Architecture



Server Acknowledgement

Accepted Trade-Offs

Parameter	Trade-off
Battery	Faster battery consumption.
Heat	Increased CPU and network activity may increase temperature.
Privacy	More frequent location collection requires stronger access controls.
Cost	Higher mobile-data and server-processing costs.
Reliability	Improved emergency delivery priority.

Conclusion:

The company should investigate an adaptive emergency-tracking policy. It prioritises reliable delivery during emergencies while retaining battery-saving behaviour during normal operation.

Task 6.7 – Technical Reflection (1 Mark)

Reflection (Within 150 Words)

Different power-saving techniques work under different operating conditions. In my prototype, charging on Wi-Fi allowed immediate uploads with a 10-second sampling interval. When battery level decreased to 20% on LTE, the policy increased the sampling interval to 120 seconds and deferred non-urgent uploads. When the network was unavailable, records were stored locally until connectivity returned. A hot-device condition also increased the sampling interval to reduce processing activity. These results demonstrate that a single power-management strategy cannot satisfy every application requirement. A tracking application, gaming application and enterprise application have different performance, connectivity and energy requirements. Therefore, mobile operating systems must provide multiple power-management mechanisms, and applications must select suitable policies according to their current operating conditions.

FINAL CONCLUSION

The Assignment 4B experiments and designs demonstrate how mobile operating systems manage application processes, memory, runtime execution, security and battery consumption.

Task 1 examined Android and iOS architecture and process management. Task 2 compared single-threaded and multi-threaded execution. Task 3 investigated ART compilation, garbage collection and runtime failures. Task 4 developed a security audit using threat modelling and Android security mechanisms.

Task 5 examined DVFS, Doze mode, wakelocks and batching. Finally, Task 6 developed a battery-aware and secure field-tracking prototype using adaptive sampling, encrypted storage and synchronisation policies.

The overall result demonstrates that mobile application engineering requires more than successful execution. Applications must be measured, tested under different conditions, protected against security threats and optimised for performance and energy efficiency.